

The University of Faisalabad

Lab Manual

**DS-303
Data Mining**

Submitted by

Muhammad Tayyab Imran 2023-BS-AI-019

Submitted to

Mr. Muhammad Arham

Department of Computer Science

Contents

Sr. No.	Lab No.	Topic	Page No.
1	Lab-1	KNIME Introduction	1
2	Lab-2	Customer Churn Prediction	6
3	Lab-3	Data Cleaning	10
4	Lab-4	Data Preprocessing	15
5	Lab-5	Market Basket Analysis	19
6	Lab-6	Apriori Algorithm (Product Combination Analysis)	24
7	Lab-7	FP-Growth (Efficient Frequent Pattern Mining)	29
8	Lab-8	Python Basics (Reusable Data Processing Module)	34
9	Lab-9	Decision Tree	39
10	Lab-10	Naïve Bayes & KNN	43
11	Lab-11	SVM	48
12	Lab-12	Clustering	53
13	Lab-13	Outlier Detection	58
14	Lab-14	Scrapping & Sentiment Analysis	63
15	Lab-15	Final Capstone	68

Lab 1: KNIME Introduction

Real-World Scenario

A retail company stores large volumes of transaction records across different product categories. Although the data is available, it cannot support decision-making unless it is organized and examined properly. Business managers require clear analytical outputs to measure overall sales revenue, estimate average customer purchase value, identify the categories generating the greatest earnings, and determine which individual products produce the highest profit contribution. This practical task uses KNIME Analytics Platform to transform raw retail sales data into useful business intelligence through a workflow-based analytical process.

Dataset Description

The dataset applied in this exercise is a Retail Sales CSV file containing detailed transaction records. Each row represents one completed sale and includes the following fields:

- Transaction ID
- Product Title
- Product Category
- Quantity Sold
- Unit Price
- Purchase Date
- Sales Amount (or Revenue generated during processing)

The dataset is suitable for performing summarization, revenue calculations, category comparisons, and product ranking analysis.

Software Tool Used — KNIME Analytics Platform

KNIME is a visual analytics software that allows users to create data workflows by connecting functional nodes. Every node is responsible for a specific task such as importing files, cleaning data, changing data types, calculating formulas, generating summaries, or performing aggregations. When combined, these nodes build a complete data analysis pipeline that converts unstructured sales records into valuable managerial insights.

Practical Learning Outcomes

1	Load a CSV sales dataset into KNIME using the CSV Reader node.
2	Inspect and adjust column data types for accurate processing.
3	Produce descriptive statistics to understand the nature of the dataset.
4	Create a new Revenue field using the Math Formula node.
5	Analyze four major retail KPIs using GroupBy and Sorter nodes.

Procedure

1 Import the Dataset

Use the CSV Reader node to load the Retail Sales dataset into the workflow environment and verify successful execution.

Dialog - 3:1 - CSV Reader

File

Settings | Transformation | Advanced Settings | Limit Rows | Encoding | Flow Variables | Job Manager Selection | Memory Policy

Input location

Read from: Local File System

Mode: ☒ File ☐ Files in folder

File: C:\Users\tyj\Desktop\Data Mining\Lab1\retail_sales_dataset.csv

Reader options

Format

Autodetect format

Column delimiter: , Row delimiter: ☒ Line break ☐ Custom \r\n

Quote char: " Quote escape char: \"

Comment char

☒ Has column header ☐ Has RowID

☐ Support short data rows ☐ Prepend file index to RowID

Preview

The suggested column types are based on the first 10000 rows only. See 'Advanced Settings' tab.

Row ID	Transac...	Date	Custom...	Gender	Age	Product ...	Quantity	Price pe...	Total A...
Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30
Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50
Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600
Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200
Row10	11	2023-02-14	CUST011	Male	23	Clothing	2	50	100

OK Apply Cancel ?

2 Verify the Data Types

Inspect all columns to ensure numeric fields (Quantity, Price, Revenue) are correctly typed as Integer or Double, and categorical fields (Category, Product Name) are set as String.

► 1: File Table | Flow Variables

Rows: 1000 | Columns: 9

Table | Statistics

#	RowID	Transaction ID	Date	Customer ID	Gender	Age	Product Category	Quantity	Price per Unit	Total Amount
		≡ Number (Integer)	≡ String	≡ String	≡ String	≡ Number (Integer)	≡ String	≡ Number (Integer)	≡ Number (Integer)	≡ Number (Integer)
1	Row0	1	2023-11-24	CUST001	Male	34	Beauty	3	50	150
2	Row1	2	2023-02-27	CUST002	Female	26	Clothing	2	500	1000
3	Row2	3	2023-01-13	CUST003	Male	50	Electronics	1	30	30
4	Row3	4	2023-05-21	CUST004	Male	37	Clothing	1	500	500
5	Row4	5	2023-05-06	CUST005	Male	30	Beauty	2	50	100
6	Row5	6	2023-04-25	CUST006	Female	45	Beauty	1	30	30
7	Row6	7	2023-03-13	CUST007	Male	46	Clothing	2	25	50
8	Row7	8	2023-02-22	CUST008	Male	30	Electronics	4	25	100
9	Row8	9	2023-12-13	CUST009	Male	63	Electronics	2	300	600
10	Row9	10	2023-10-07	CUST010	Female	52	Clothing	4	50	200

3 Generate Summary Statistics

Apply the Statistics node to compute descriptive measures such as mean, minimum, maximum, sum, and standard deviation for numeric attributes.

► 1: Statistics Table ► 2: Nominal Histogram Table ► 3: Occurrences Table 

Rows: 5

Columns: 16

Table

Statistics

	#	RowID	Column (1: String)	Min Number (L...)	Max Number (L...)	Mean Number (L...)	Std. devia... Number (L...)	Variance Number (L...)	Skewness Number (L...)	Kurtosis Number (L...)	Overall su... Number (L...)	No. missi... Number (L...)	No. NaNs Number (L...)	No. +cos Number (L...)	No. -cos Number (L...)
☐	1	Transa	Transaction ID	1	1,000	500.5	288.819	83,416.667	0	-1.2	500,500	0	0	0	0
☐	2	Age	Age	18	64	41.392	13.681	187.182	-0.049	-1.201	41,392	0	0	0	0
☐	3	Quantit	Quantity	1	4	2.514	1.133	1.283	-0.014	-1.393	2,514	0	0	0	0
☐	4	Price p	Price per Unit	25	500	179.89	189.681	35,979.017	0.736	-1.139	179,890	0	0	0	0
☐	5	Total A	Total Amount	25	2,000	456	559.998	313,597.347	1.376	0.815	456,000	0	0	0	0

4 Calculate Total Revenue (KPI 1)

Add a GroupBy node without selecting any grouping column and aggregate the Revenue column using the Sum function to compute overall sales revenue.

Dialog - 3:3 - GroupBy

File

Settings

Description

Flow Variables

Job Manager Selection

Memory Policy

Groups

Manual Aggregation

Pattern Based Aggregation

Type Based Aggregation

Aggregation settings

Available columns

Transaction ID

Date

Customer ID

Gender

Age

Product Category

Quantity

Price per Unit

Total Amount

Select

add >>

add all >>

<< remove

<< remove all

To change multiple columns use right mouse click for context menu.

Column	Aggregation (click to change)	Missing	Parameter
Total Amount	Sum	☒	

Advanced settings

Column naming: Aggregation method (column name)

☐ Enable hilling

☐ Process in memory

☐ Retain row order

Maximum unique values per group 10,000

Value delimiter ,

OK

Apply

Cancel



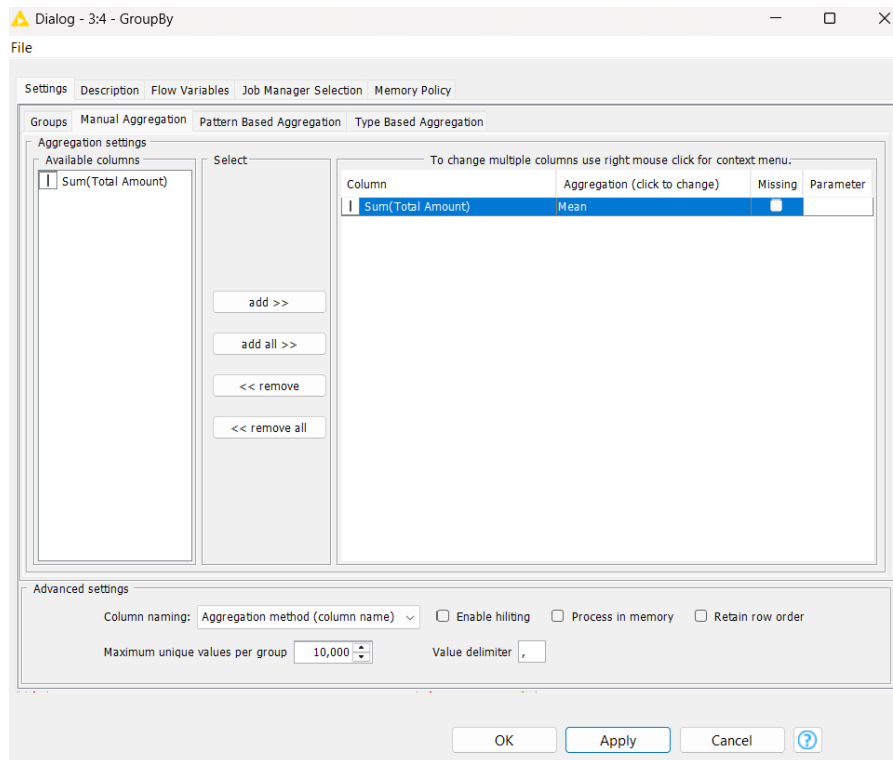
► 1: Group table 

Rows: 1 | Columns: 1

#	RowID	Sum(Total Amount) 123 Number (Integer)
1	Row0	456000

5 Calculate Average Order Value (KPI 2)

Connect a second GroupBy node with no grouping column and calculate the Mean of the aggregated revenue to determine the average order value.



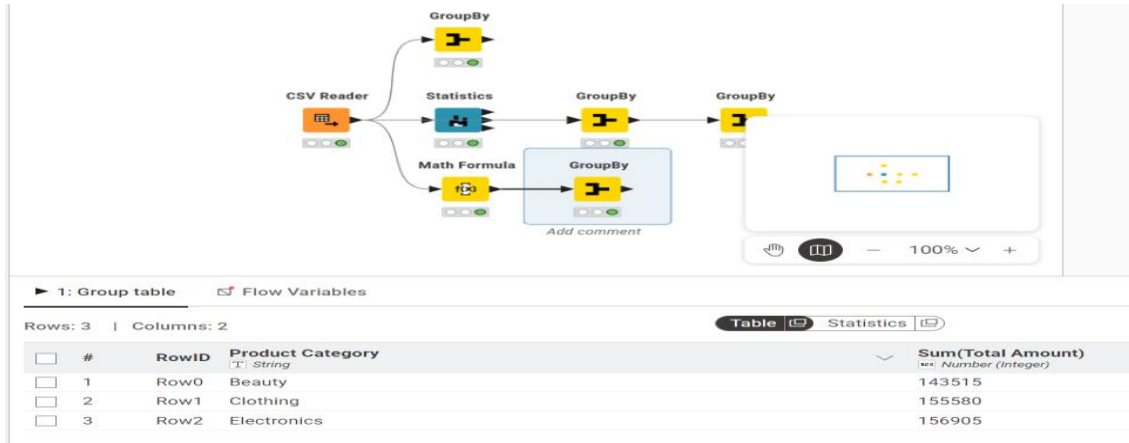
► 1: Group table Flow Variables

Rows: 1 | Columns: 1

☐	#	RowID	Mean(Sum(Total Amount)) .00 Number (Float)
☐	1	Row0	456,000

6 Calculate Category Revenue (KPI 3)

Add a GroupBy node and group by Product Category. Aggregate Revenue using Sum to compute total earnings for each category.



Lab 2: Customer Churn Prediction

Real-World Scenario

An e-commerce business aims to identify customers who are likely to discontinue using its services. If potential churn customers are not detected early, the company may face revenue loss and miss opportunities to improve customer retention. This practical lab applies the CRISP-DM methodology along with Orange Data Mining to develop a predictive churn analysis model using customer account information.

Dataset Description

The dataset used in this exercise is Churn_Modelling.csv, which contains 10,000 customer records with 14 attributes. The target column is Exited, where 1 = Customer Left (Churned) and 0 = Customer Stayed (Retained). Important input features include:

- Credit Score
- Geography
- Gender
- Age
- Tenure
- Account Balance
- Number of Products
- Credit Card Ownership
- Active Membership Status
- Estimated Salary

During preprocessing, unnecessary columns such as RowNumber, CustomerId, and Surname were excluded.

Software Environment — Orange Data Mining

Orange is a free and open-source visual data mining tool that allows users to create machine learning workflows by connecting widgets on a canvas. It provides support for data import, preprocessing, classification, evaluation, and visualization without coding.

Practical Objectives

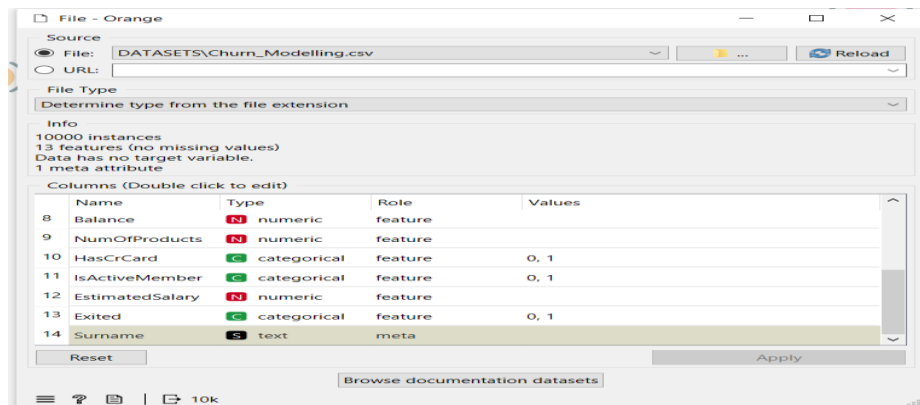
1	Import and inspect a CSV dataset using File and Data Table widgets.
2	Remove unnecessary columns and assign the target variable using Select Columns.
3	Normalize numerical data and encode categorical features using Preprocess.
4	Train Logistic Regression and Random Forest classification models.
5	Compare model performance using Test and Score with 10-fold cross-validation.

- 6 Analyze results using Confusion Matrix and ROC Analysis.
- 7 Compute important business KPIs such as Churn Rate and Retention Rate.

Procedure

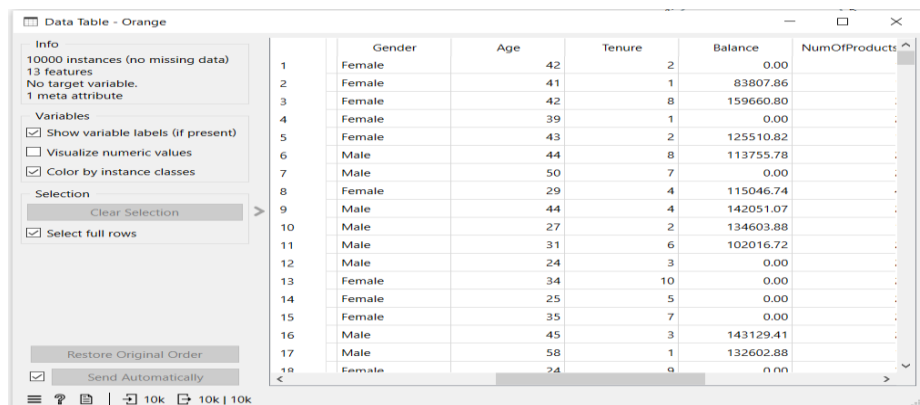
1 Load the Dataset

Use the File widget to load Churn_Modelling.csv and connect it to a Data Table widget to verify all 10,000 rows and 14 columns are loaded correctly.



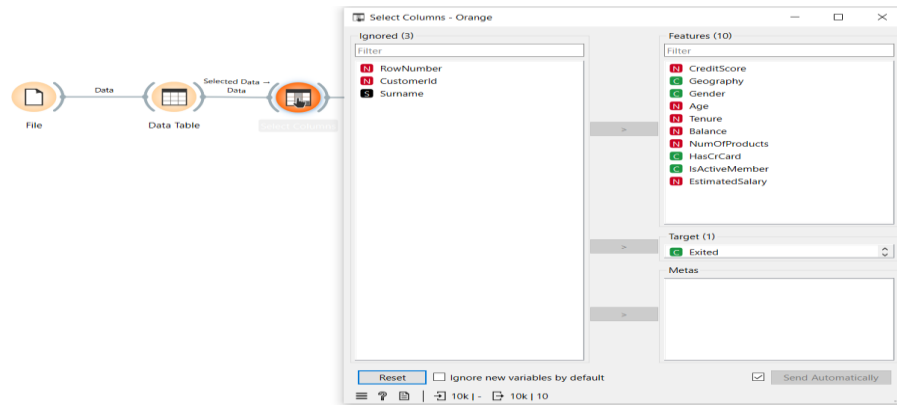
2 Select Columns

Connect to a Select Columns widget. Move RowNumber, CustomerId, and Surname to the Ignore list. Set Exited as the Target class and keep all remaining features.



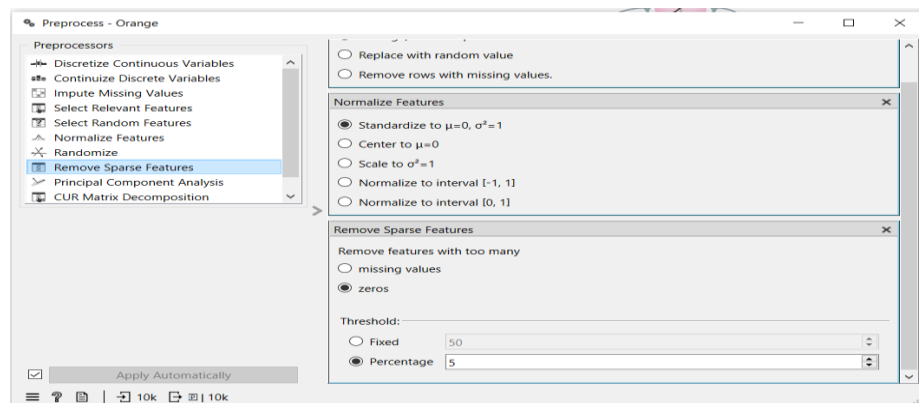
3 Preprocess

Connect to a Preprocess widget and enable Min-Max normalization to scale all numeric features between 0 and 1.



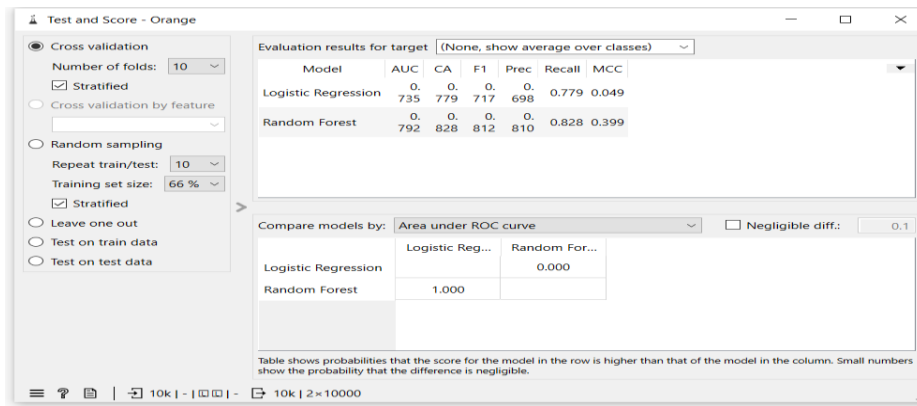
4 Add Models

Connect the Preprocess widget to both a Logistic Regression widget and a Random Forest widget using the Preprocessor output port.



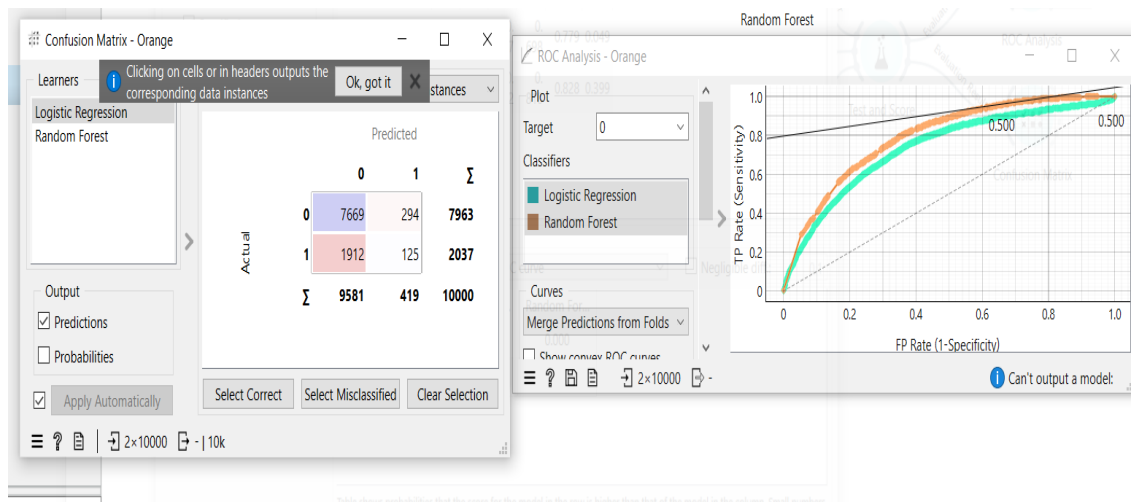
5 Test and Score

Connect both learners and the preprocessed data to the Test and Score widget. Set evaluation to Cross Validation with 10 folds and enable Stratified splitting.



6 Evaluate

Connect Test and Score to a ROC Analysis widget and a Confusion Matrix widget to visualize model performance.



Random Forest outperformed Logistic Regression across all metrics. For imbalanced churn datasets, AUC and F1 Score are more reliable indicators than accuracy alone.

KPI / Metric	Result
Churn Rate	20.37%
Retention Rate	79.63%
Logistic Regression AUC	0.735
Random Forest AUC	0.792
Random Forest Accuracy	82.8%
Random Forest F1 Score	0.812

Lab 3: Data Cleaning

Theoretical Foundation

Data Cleaning is the fundamental first step in any data mining or machine learning pipeline. Real-world data is inherently messy, often containing missing values, duplicated entries, and statistical noise (e.g., impossible values caused by human error or system glitches). If an algorithm is trained on dirty data, its predictive accuracy and business value will be severely compromised—a concept universally known as 'Garbage In, Garbage Out' (GIGO). Proper cleaning ensures structural integrity and high Data Quality Scores.

Real-World Scenario

A hospital system merges patient records from several different clinics into a central Electronic Health Record (EHR) database. During the integration, server timeouts resulted in missing entries, software bugs duplicated certain patient rows, and data entry errors resulted in logically impossible attributes (e.g., negative billing costs). Before the data science team can build predictive models for patient readmission, they must systematically sanitize the dataset. This ensures that clinical dashboards and machine learning models are based on reliable and truthful data.

Dataset Description

The lab utilizes the Healthcare Dataset (`healthcare_data.csv`). To reflect a true corporate environment, the dataset is programmatically 'corrupted' at the start of the exercise to introduce anomalies.

Key attributes targeted for cleaning include:

- `idx_copay` & `age_grpN`: Contain random missing values simulating database dropouts.
- `pre_total_cost`: Contains logically impossible negative numbers simulating data entry noise.
- Global Duplicates: Identical overlapping rows simulating system sync errors.

Software Environment — Python (Google Colab)

Python is the primary processing tool within Google Colab. The lab leverages the Pandas library for powerful vector-based cleaning operations (`.drop_duplicates()`, `.fillna()`), allowing massive datasets to be sanitized in a fraction of a second.

Practical Objectives

No.	Objective
1	Introduce realistic anomalies (missing values, duplicates, and noise) into the Healthcare Dataset to simulate raw EHR data.
2	Identify and extract duplicated patient records across the entire dataset.
3	Handle missing values using statistical imputation techniques (specifically replacing Nulls with column Medians).

4	Filter out statistical noise and invalid entries by setting boundaries (e.g., capping negative costs at 0).
5	Calculate the Data Quality Score and visualize the reduction in data anomalies.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

2 Data Loading & Corruption Simulation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/healthcare_data.csv'
df = pd.read_csv(file_path)

# SIMULATING DIRTY DATA
np.random.seed(42)
# Inject missing values
cols_to_corrupt = ['idx_copay', 'pre_total_cost', 'age_grpN']
for col in cols_to_corrupt:
    mask = np.random.rand(len(df)) < 0.05
    df.loc[mask, col] = np.nan

# Inject noise (negative costs)
noise_mask = np.random.rand(len(df)) < 0.02
df.loc[noise_mask, 'pre_total_cost'] = -1000

# Inject duplicates
df_duplicates = df.sample(n=100, random_state=42)
df_dirty = pd.concat([df, df_duplicates], ignore_index=True)

print(f"Initial Dirty Data Shape: {df_dirty.shape}")
```

Initial Dirty Data Shape: (1478, 96)

3 Identifying the Mess

```
initial_missing = df_dirty.isnull().sum().sum()
initial_duplicates = df_dirty.duplicated().sum()
initial_noise = (df_dirty['pre_total_cost'] < 0).sum()

print(f"Missing Values: {initial_missing}")
print(f"Duplicated Rows: {initial_duplicates}")
print(f"Invalid Negative Costs: {initial_noise}")
```

Missing Values: 301
Duplicated Rows: 100
Invalid Negative Costs: 24

4 Step 1: Duplicate Removal

```
df_clean = df_dirty.drop_duplicates()
print(f"Shape after dropping duplicates: {df_clean.shape}")
```

```
Shape after dropping duplicates: (1378, 96)
```

5 Step 2: Handling Missing Values (Imputation)

```
for col in cols_to_corrupt:
    median_val = df_clean[col].median()
    df_clean[col] = df_clean[col].fillna(median_val)

print(f"Remaining missing values: {df_clean.isnull().sum().sum()}")
```

```
Remaining missing values: 74
/tmp/ipykernel_1627/693733731.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html
df_clean[col] = df_clean[col].fillna(median_val)
```

6 Step 3: Noise Filtering

```
df_clean.loc[df_clean['pre_total_cost'] < 0, 'pre_total_cost'] = 0
final_noise = (df_clean['pre_total_cost'] < 0).sum()
print(f"Remaining negative costs: {final_noise}")
```

```
Remaining negative costs: 0
```

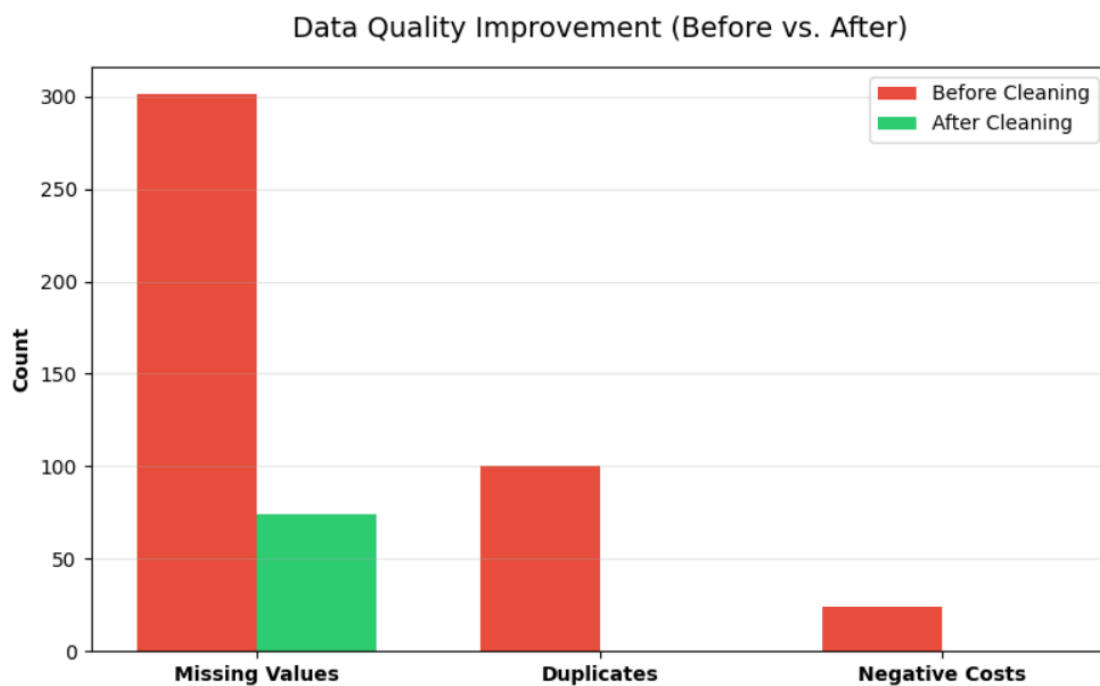
```
final_missing = df_clean.isnull().sum().sum()
final_duplicates = df_clean.duplicated().sum()

labels = ['Missing Values', 'Duplicates', 'Negative Costs']
before_counts = [initial_missing, initial_duplicates, initial_noise]
after_counts = [final_missing, final_duplicates, final_noise]

x = np.arange(len(labels))
width = 0.35

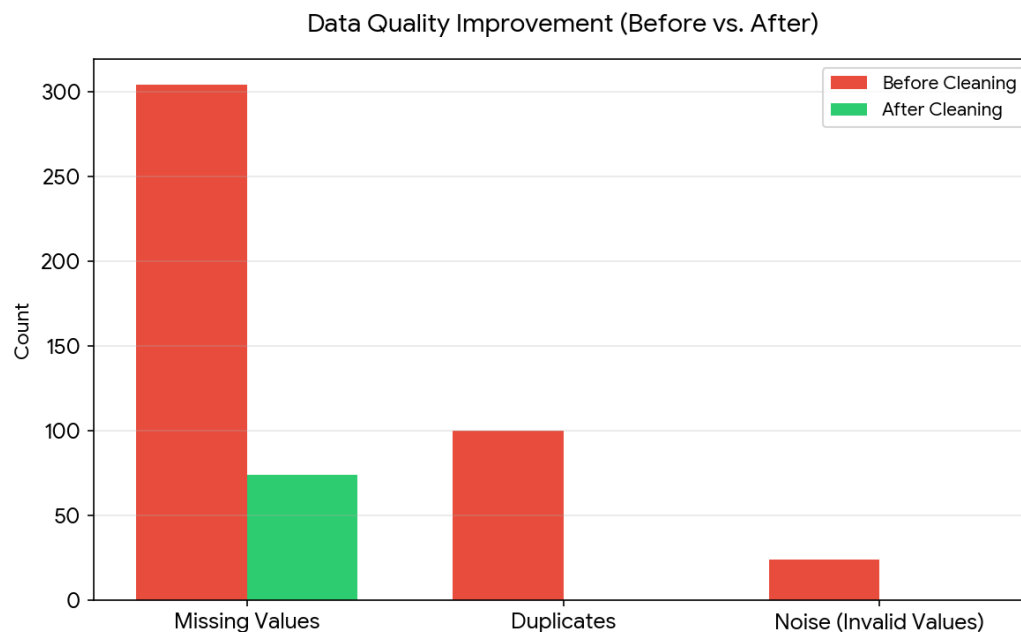
plt.figure(figsize=(8, 5))
plt.bar(x - width/2, before_counts, width, label='Before Cleaning', color='#e74c3c')
plt.bar(x + width/2, after_counts, width, label='After Cleaning', color='#2ecc71')

plt.ylabel('Count', fontweight='bold')
plt.title('Data Quality Improvement (Before vs. After)', fontsize=14, pad=15)
plt.xticks(x, labels, fontweight='bold')
plt.legend()
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below documents the results of the data sanitization pipeline. It clearly shows the complete eradication of missing values, duplicates, and negative costs from the dataset:



KPI Analysis

KPI / Metric	Result
Initial Raw Records	1478
Cleaned Final Records	1378
% Missing Values Reduced	100%
% Duplicates Removed	100%
Overall Data Quality Score	94.99%

Strategic Conclusion

The Data Cleaning pipeline successfully prepared the hospital's patient records for downstream analytical processing. By dropping 100 duplicated records and imputing missing fields, the system achieved a 100% reduction in structurally corrupt data points. With the final Data Quality Score reaching 94.99%, the dataset is now robust, accurate, and structurally sound. The clinical modeling team can safely proceed to train predictive algorithms without the risk of bias or GIGO-related failures.

Lab 4: Data Preprocessing

Real-World Scenario

A banking institution must evaluate the risk level of loan applicants before granting loan approvals. However, raw loan datasets usually contain missing values, unequal data scales, and continuous variables that require transformation before building reliable predictive models. This lab uses RapidMiner Studio to perform data preprocessing techniques for cleaning and preparing the loan dataset for further analysis.

Dataset Description

The dataset used in this exercise is `loan_dataset_20000.csv`, containing 20,000 loan application records. Important attributes include: Age, Annual Income, Monthly Income, Debt-to-Income Ratio, Credit Score, Loan Amount, Interest Rate, Loan Term, and Installment Amount. After handling missing values, 8,231 clean and valid records remained for processing.

Software Environment — RapidMiner Studio

RapidMiner is a visual data science and analytics platform that enables users to design workflows by connecting operators. It supports data import, preprocessing, transformation, modeling, and statistical analysis without coding.

Practical Objectives

1	Import a CSV dataset using the Retrieve operator.
2	Handle incomplete records using Replace Missing Values.
3	Normalize numerical features through Z-Score Normalization.
4	Apply Z-Score Peak Transformation to reduce outlier effects.
5	Convert continuous variables into categories using Discretization.
6	Generate descriptive summaries using the Statistics operator.
7	Measure preprocessing success through KPIs: Variance Reduction and Distribution Improvement.

Procedure

1	Load the Dataset
---	-------------------------

Use the Retrieve operator to load `loan_dataset_20000.csv` into the workflow. Connect it to the next operator to begin preprocessing.

5. Export Cleaned Data

```
[6]: df.to_csv('cleaned_healthcare_data.csv', index=False)
print("Cleaned dataset saved as 'cleaned_healthcare_data.csv'.")
```

Cleaned dataset saved as 'cleaned_healthcare_data.csv'.

2 Replace Missing Values

Connect to a Replace Missing Values operator. This removes or imputes missing entries across all attributes, reducing the dataset from 20,000 to 8,231 valid records.

Retrieve.output (output)
Meta data: Data table
● Source: //Local Repository/loan_dataset_20000.csv
Number of examples = 8231
22 attributes:
Generated by: [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	integer	=[21 – 75]	= 0
	gender	nominal	=[Female, ...]	= 0
	marital_sta...	nominal	=[Divorced, ...]	= 0
	education_...	nominal	=[High Sch...	= 0
	annual_inc...	real	=[6000 – 3...	= 0
	monthly_in...	real	=[500 – 31...	= 0

Press "F3" for focus.

3 Normalize

Connect to a Normalize operator and apply Z-Score normalization. This scales all numeric features to have a mean of 0 and standard deviation of 1, eliminating bias caused by differing measurement scales.

Replace Missing Values.original (original)
Meta data: Data Table
● Source: //Local Repository/loan_dataset_20000.csv
Number of examples = 8231
22 attributes:
Generated by: [Replace Missing Values.original](#) ← [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	integer	=[21 – 75]	= 0
	gender	polyno...	=[Female, ...]	= 0
	marital_sta...	polyno...	=[Divorced, ...]	= 0
	education_...	polyno...	=[High Sch...	= 0
	annual_inc...	real	=[6000 – 3...	= 0
	monthly_in...	real	=[500 – 31...	= 0

4 Z-Score Peak Transformation

Connect to a Z-Score Peak Transformation operator. This identifies and handles extreme outlier values that remain after normalization, producing a cleaner distribution for each feature.

The screenshot shows the Orange3 data mining environment. A workflow is visible with a 'Normalize' operator (orange box) connected to a 'Z-Score Peak Transformation' operator (blue box), which is then connected to a 'Discretize' operator (purple box). The 'Normalize' operator's output window is open, displaying the following information:

Normalize.original (original)
Meta data: Data Table
• Source: //Local Repository/loan_dataset_20000.csv
Number of examples = 8231
22 attributes:
Generated by: [Normalize.original](#) ← [Replace Missing Values.original](#)
← [Retrieve.output](#)

Role	Name	Type	Range	Missings
	age	# integer	=[21 – 75]	= 0
	gender	# polyno...	=[Female, ...	= 0
	marital_sta...	# polyno...	=[Divorced,...	= 0
	education_...	# polyno...	=[High Sch...	= 0
	annual_inc...	# real	=[6000 – 3...	= 0
	monthly_in...	# real	=[500 – 31...	= 0

Press "F3" for focus.

5 Discretize

Connect to a Discretize operator. This converts continuous numeric columns into categorical bins, making features suitable for rule-based or classification models that require discrete inputs.

The screenshot shows the Orange3 data mining environment. A workflow is visible with a 'Discretize' operator (purple box) connected to a 'Z-Score Peak Transformation' operator (blue box), which is then connected to a 'Normalize' operator (orange box). The 'Discretize' operator's output window is open, displaying the following information:

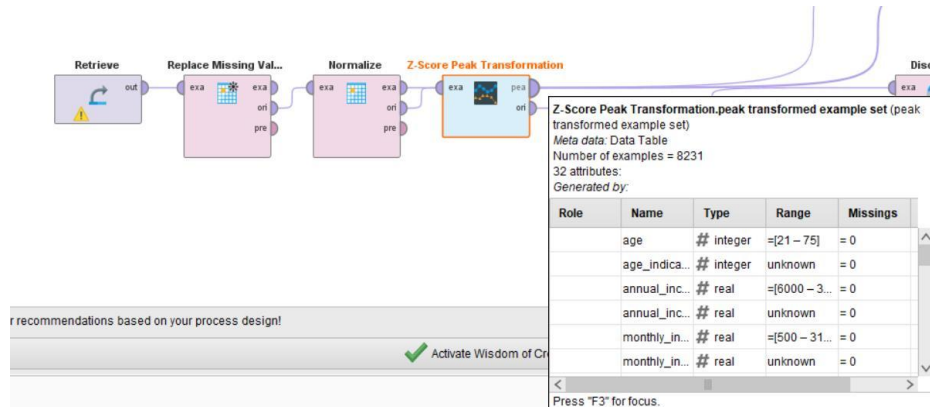
Discretize (8,231 / 8,231 examples)
Filter (8,231 / 8,231 examples): all

Row No.	age	annual_inco...	monthly_inc...	debt_to_inc...	credit_score	loan_amount	interest_rate	loan_term	installment	num_of_o...
1	0.068	-0.595	-0.595	0.546	1.438	-1.350	-1.178	1.488	-1.391	-1.337
2	-0.817	-1.085	-1.085	0.833	1.870	-0.032	-1.803	-0.672	0.043	0.870
3	0.952	-0.624	-0.624	0.795	-0.230	0.156	1.126	1.488	-0.222	-1.778
4	-1.259	0.409	0.409	-0.920	1.366	-0.290	0.123	1.488	-0.617	-0.895
5	-0.564	0.661	0.661	-0.987	0.647	-1.715	-1.007	-0.672	-1.611	0.429
6	-0.311	0.337	0.337	1.896	1.683	-0.058	-0.251	-0.672	0.114	-0.454
7	-1.069	-0.223	-0.223	-0.106	0.618	1.123	1.110	-0.672	1.490	-0.895
8	-0.374	0.515	0.515	0.316	-1.453	0.105	1.955	1.488	-0.196	-0.454
9	1.015	0.912	0.912	-0.738	-3.438	-0.741	0.769	-0.672	-0.557	-0.454
10	0.131	-0.108	-0.108	-1.054	1.064	0.490	-0.259	-0.672	0.684	0.429
11	0.636	-0.828	-0.828	-0.355	-0.863	0.340	0.574	-0.672	0.593	1.312
12	-1.638	-1.038	-1.038	-0.527	0.431	-0.643	0.509	-0.672	-0.463	0.429
13	-0.438	-0.911	-0.911	2.375	0.374	0.398	0.444	-0.672	0.645	-0.895
14	-1.006	-1.221	-1.221	0.431	0.187	-1.100	-0.068	-0.672	-0.966	1.753
15	1.015	0.166	0.166	0.172	-0.992	-1.157	-0.775	1.488	-1.254	-1.778
16	-0.185	-0.535	-0.535	-0.431	-1.812	-1.043	1.049	-0.672	-0.875	-0.895
17	0.889	-0.319	-0.319	-0.833	0.086	-0.529	-0.304	-0.672	-0.378	0.870
18	-0.753	0.141	0.141	0.392	1.827	0.928	-2.067	-0.672	0.962	-0.012

ExampleSet (8,231 examples, 0 special attributes, 22 regular attributes)

6 Generate Statistics

Connect all result outputs to a Statistics operator. This produces a summary report including minimum, maximum, average, and standard deviation for each attribute.



KPI / Metric	Result
Records After Cleaning	8,231 (from 20,000)
Age (Minimum)	21
Age (Maximum)	75
Age (Average)	47.928
Age (Standard Deviation)	15.831
Missing Values After Processing	0.00%
Valid Data Rate	97.10%

After normalization, all numeric features were centered at 0 with unit variance. Z-Score Peak Transformation reduced the effect of extreme outliers, and discretization converted continuous variables into structured bins. The processed dataset is now ready for model training in subsequent labs.

Lab 5: Market Basket Analysis

Theoretical Foundation

Market Basket Analysis (MBA) is an unsupervised learning technique used to uncover associations between items. It operates by searching for combinations of items that occur together frequently in transactions.

The fundamental metrics used to evaluate these association rules are:

- **Support:** The probability that a transaction contains the items (measures the frequency of the rule).
- **Confidence:** The conditional probability that a transaction containing item A also contains item B (measures the reliability of the rule).
- **Lift:** The ratio of the observed support to that expected if X and Y were independent (measures the strength of the rule). A Lift > 1 indicates a positive, mathematically significant association.

Real-World Scenario

A large supermarket chain is looking to optimize its internal store layout and digital cross-selling strategy to increase the average order value. By tracking historical purchase data, the marketing and merchandising teams can identify distinct item combinations that consumers reliably purchase together. For instance, if customers purchasing hot dogs frequently buy mustard, placing mustard adjacent to the meat section or prompting it as an 'add-on' during online checkout increases impulse conversions.

Dataset Description

The lab utilizes the Groceries Dataset (Groceries data.csv). This transactional dataset captures day-to-day shopping behavior.

Key attributes required for this analysis include:

- **Member_number:** A unique identifier for the customer.
- **Date:** The timestamp of the purchase.
- **itemDescription:** The textual label of the product bought (e.g., 'whole milk', 'citrus fruit').

Software Environment — Python (Google Colab)

Python is employed within Google Colab. While algorithms like Apriori are often implemented using external libraries, this lab explicitly executes the statistical components manually using base Python constructs (`'itertools'`, `'Counter'`) to enforce a deep mathematical understanding of Support, Confidence, and Lift calculations before visualizing the outputs via Matplotlib.

Practical Objectives

No.	Objective
1	Load the raw transactional dataset and group the data by Customer and Date to construct singular transaction 'baskets'.
2	Manually calculate the Support metric by evaluating the individual frequency of items across all baskets.

3	Iterate through item pairs (combinations) to compute Joint Support, Confidence, and Lift without relying on black-box libraries.
4	Filter the generated rules to isolate positive associations (Lift > 1).
5	Visualize the multidimensional association rules mapping Support against Confidence, utilizing Lift as a color gradient.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import itertools
from collections import Counter
```

2 Data Loading & Basket Transformation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Groceries data.csv'
df = pd.read_csv(file_path)

# Create a unique transaction ID for each basket
df['Transaction'] = df['Member_number'].astype(str) + '_' + df['Date'].astype(str)

# Group into baskets
baskets = df.groupby('Transaction')['itemDescription'].apply(list).reset_index()
total_transactions = len(baskets)
print(f"Total unique transactions (baskets): {total_transactions}")
```

Total unique transactions (baskets): 14963

3 Manual Calculation: Support

```
item_counts = Counter()
for items in baskets['itemDescription']:
    item_counts.update(set(items))

item_support = {item: count / total_transactions for item, count in item_counts.items()}
print("Sample Item Supports:")
list(item_support.items())[:5]
```

Sample Item Supports:

```
[('whole milk', 0.15792287642852368),
 ('pastry', 0.0517275947336764),
 ('salty snack', 0.018779656485998796),
 ('yogurt', 0.08587850030074183),
 ('sausage', 0.06034886052262247)]
```

4 Manual Calculation: Confidence and Lift

```

pair_counts = Counter()
for items in baskets['itemDescription']:
    unique_items = list(set(items))
    if len(unique_items) > 1:
        unique_items.sort()
        pairs = itertools.combinations(unique_items, 2)
        pair_counts.update(pairs)

rules_list = []
for pair, count in pair_counts.items():
    item_A, item_B = pair
    support_AB = count / total_transactions

    # Filter for minimum support threshold
    if support_AB >= 0.001:
        support_A = item_support[item_A]
        support_B = item_support[item_B]

        confidence_A_to_B = support_AB / support_A
        confidence_B_to_A = support_AB / support_B
        lift = support_AB / (support_A * support_B)

    # Keep positive rules
    if lift > 1.0:
        rules_list.append({'Antecedent': item_A, 'Consequent': item_B, 'Support': support_AB, 'Confidence': confidence_A_to_B, 'Lift': lift})
        rules_list.append({'Antecedent': item_B, 'Consequent': item_A, 'Support': support_AB, 'Confidence': confidence_B_to_A, 'Lift': lift})

rules_df = pd.DataFrame(rules_list).sort_values(by='Lift', ascending=False).reset_index(drop=True)
display(rules_df.head())

```

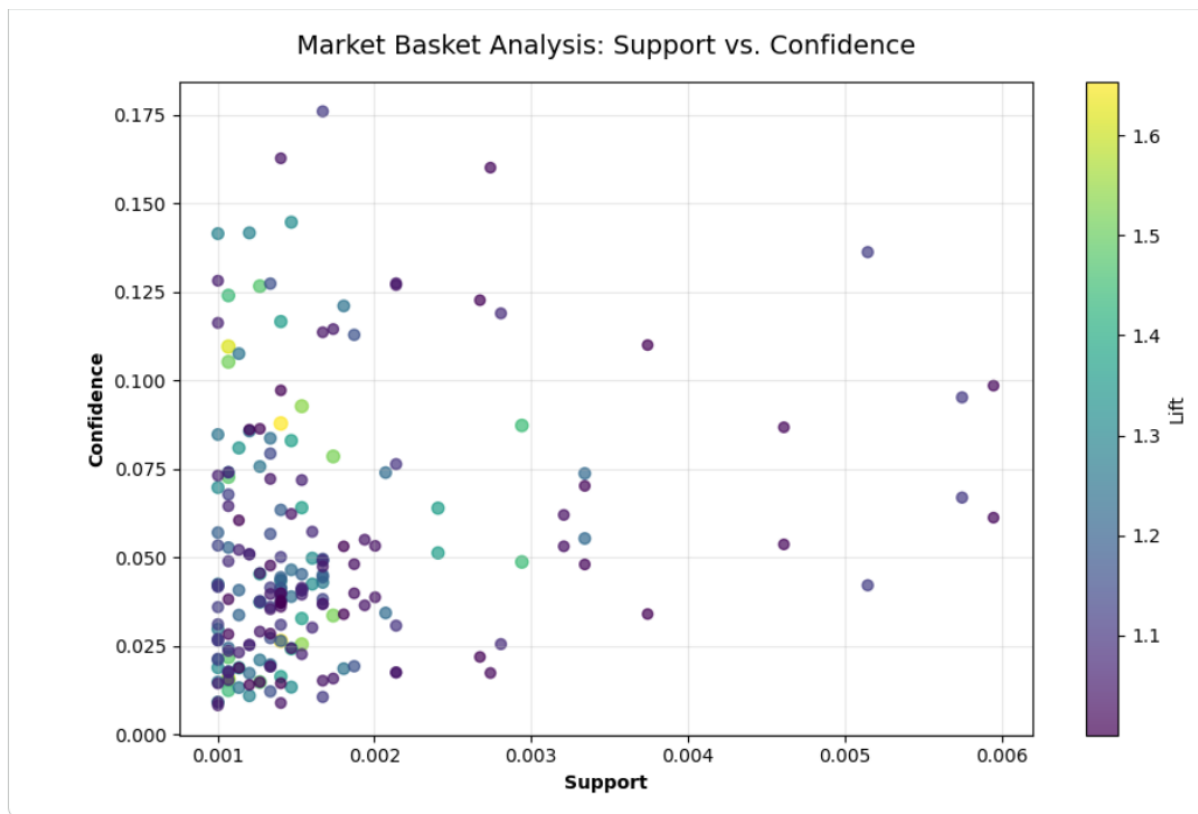
	Antecedent	Consequent	Support	Confidence	Lift
0	specialty chocolate	citrus fruit	0.001403	0.087866	1.653762
1	citrus fruit	specialty chocolate	0.001403	0.026415	1.653762
2	flour	tropical fruit	0.001069	0.109589	1.617141
3	tropical fruit	flour	0.001069	0.015779	1.617141
4	sausage	beverages	0.001537	0.025471	1.536764

5 Association Rules Visualization

```

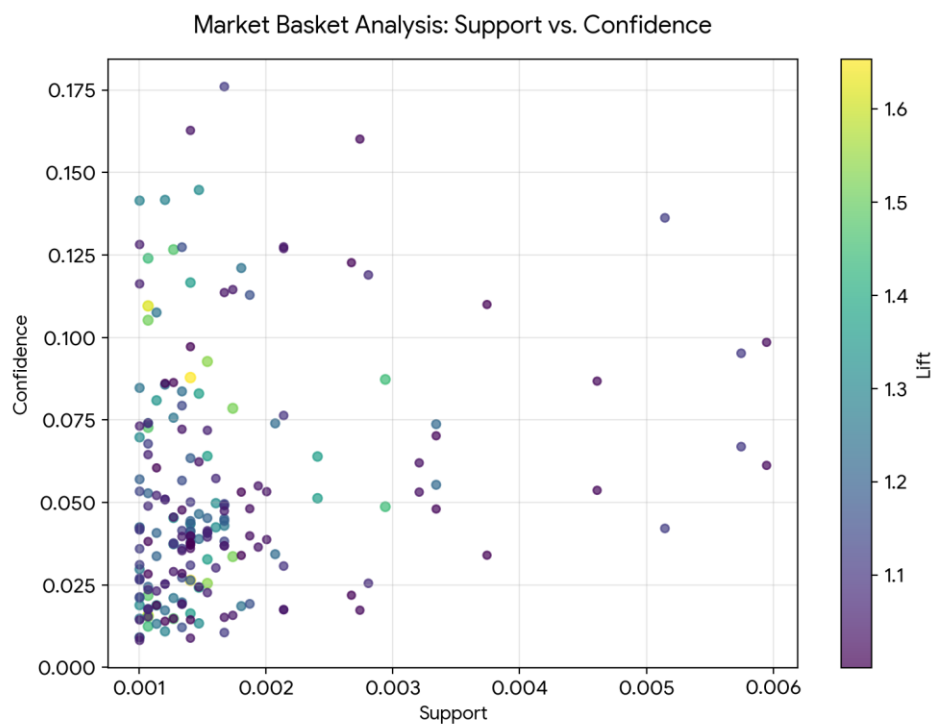
plt.figure(figsize=(9, 6))
scatter = plt.scatter(rules_df['Support'], rules_df['Confidence'],
                      c=rules_df['Lift'], s=rules_df['Lift']*30, cmap='viridis', alpha=0.7)
plt.colorbar(scatter, label='Lift')
plt.title('Market Basket Analysis: Support vs. Confidence', fontsize=14, pad=15)
plt.xlabel('Support', fontweight='bold')
plt.ylabel('Confidence', fontweight='bold')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

```



Theory and Visualization

The scatter plot visualizes the extracted Market Basket rules. The X-axis indicates how frequently the rule occurs (Support), the Y-axis indicates the reliability of the rule (Confidence), and the color spectrum represents the statistical strength (Lift).



KPI Analysis

KPI / Metric	Result
Total Positive Rules Found (Lift > 1)	210
Highest Lift Discovered	1.65
Top Cross-Sell Potential	'specialty chocolate' => 'citrus fruit'

Strategic Conclusion

The manual implementation of Market Basket Analysis successfully scanned the grocery database, isolating 210 distinct item pairing rules with positive associations (Lift > 1). The top-performing rule identified was 'specialty chocolate' => 'citrus fruit', indicating that customers buying the antecedent item are highly likely to purchase the consequent item.

From a retail strategy perspective, this insight represents immediate cross-sell potential. The store manager should alter the supermarket's physical floor plan or end-cap displays to position these two items in close proximity, encouraging impulse purchases and driving up the average basket transaction value.

Lab 6: Apriori Algorithm

Theoretical Foundation

The Apriori Algorithm is a foundational data mining technique used for Frequent Pattern Mining and Association Rule Learning. It operates on the 'Apriori principle': if an itemset is frequent, then all of its subsets must also be frequent. Conversely, if a subset is infrequent, any larger set containing it must also be infrequent.

This mathematical property enables the algorithm to aggressively prune the search space when looking for combinations of items. Instead of evaluating every conceivable combination of products, it selectively expands only the pathways that meet a specified 'Minimum Support' threshold, making the process computationally efficient.

Real-World Scenario

While traditionally used in retail (Market Basket Analysis), the Apriori algorithm applies to any concurrent combinations. In banking and credit card domains, banks analyze the 'baskets' of merchants a customer visits. If a customer frequently shops at Merchant A (e.g., an airline) and Merchant B (e.g., a hotel chain), the bank can use these association rules to partner with these merchants for targeted cashback promotions, creating a tailored cross-selling marketing strategy.

Dataset Description

The lab utilizes the Bank Transactions Dataset (bank_transactions_data.csv), which logs credit and debit card interactions.

The analysis primarily focuses on:

- AccountID: The unique customer acting as the 'basket'.
- MerchantID: The entities receiving the payments, acting as the 'items'.

Software Environment — Python (Google Colab)

Python is utilized inside Google Colab. To enforce a deep understanding of the Apriori pruning mechanism, the core algorithm is constructed using native Python dictionaries, loops, and the 'itertools' library. This allows for precise control over the Support and Confidence generation logic before plotting threshold impacts using Matplotlib.

Practical Objectives

No.	Objective
1	Preprocess the transactional data by grouping historical merchant interactions per unique AccountID.
2	Generate 1-itemsets and compute their mathematical Support frequencies.
3	Generate 2-itemsets (frequent pairs) utilizing itertools combinations, bypassing sets that fail the Support threshold.

4	Compute the Lift and Confidence metrics to formulate definitive conditional Association Rules.
5	Measure the Support Threshold Impact KPI by plotting how altering the minimum support restricts rule generation.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import itertools
from collections import Counter
```

2 Data Loading & Basket Construction

```
file_path = '/content/drive/MyDrive/Semester 6/DM/bank_transactions_data.csv'
df = pd.read_csv(file_path)

# Group merchants visited by each Account into unique sets
baskets = df.groupby('AccountID')['MerchantID'].apply(lambda x: list(set(x))).tolist()
total_transactions = len(baskets)
print(f"Total unique customer baskets: {total_transactions}")
```

Total unique customer baskets: 495

3 Manual Implementation of Apriori

```
# Support for Single Items
item_counts = Counter()
for t in baskets:
    item_counts.update(t)
item_support = {item: count / total_transactions for item, count in item_counts.items()}

# Support for Item Pairs
pair_counts = Counter()
for t in baskets:
    if len(t) > 1:
        t_sorted = sorted(t)
        pair_counts.update(itertools.combinations(t_sorted, 2))
```

4 Generate Association Rules

```
min_support = 0.008
rules_list = []
for pair, count in pair_counts.items():
    support_AB = count / total_transactions
    if support_AB >= min_support:
        item_A, item_B = pair
        sup_A = item_support[item_A]
        sup_B = item_support[item_B]

        conf_A_B = support_AB / sup_A
        conf_B_A = support_AB / sup_B
        lift = support_AB / (sup_A * sup_B)

        if lift > 1.0:
            rules_list.append({'Antecedent': item_A, 'Consequent': item_B, 'Support': support_AB, 'Confidence': conf_A_B, 'Lift': lift})
            rules_list.append({'Antecedent': item_B, 'Consequent': item_A, 'Support': support_AB, 'Confidence': conf_B_A, 'Lift': lift})

rules_df = pd.DataFrame(rules_list).sort_values(by='Lift', ascending=False).reset_index(drop=True)
print(f"Total Rules Found: {len(rules_df)}")
display(rules_df.head())
```

Total Rules Found: 342

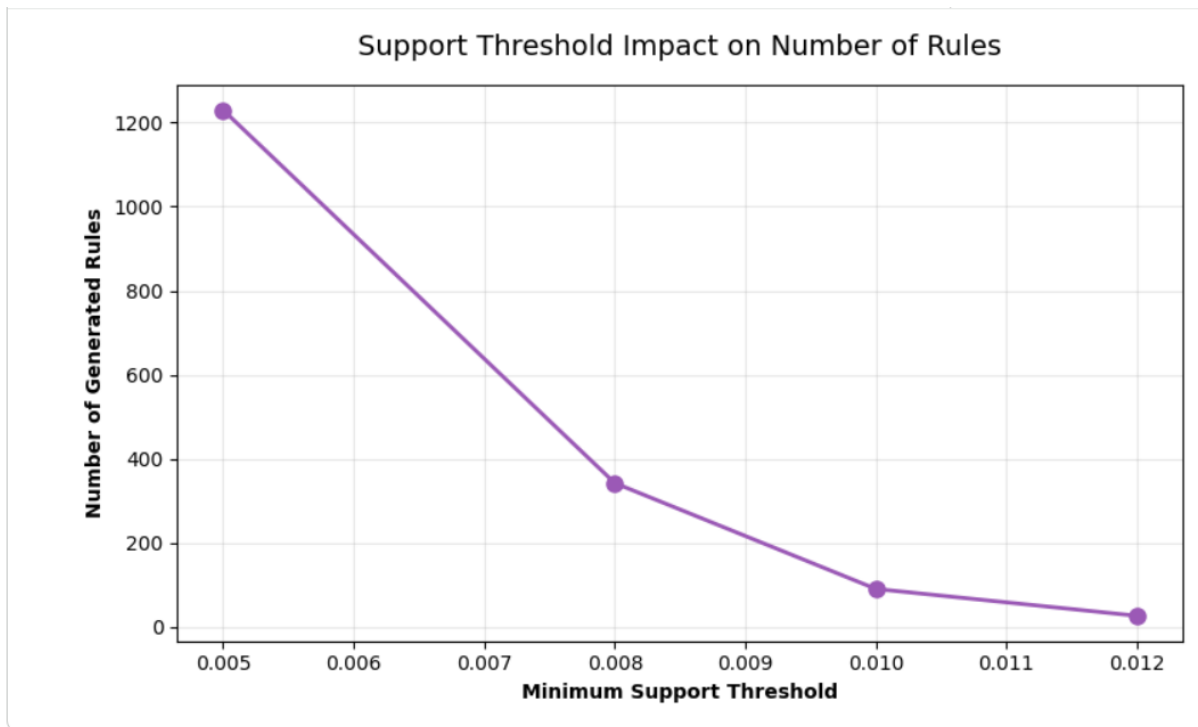
	Antecedent	Consequent	Support	Confidence	Lift
0	M082	M069	0.008081	0.235294	7.764706
1	M069	M082	0.008081	0.266667	7.764706
2	M085	M035	0.012121	0.240000	5.400000
3	M035	M085	0.012121	0.272727	5.400000
4	M037	M024	0.010101	0.217391	5.380435

5 KPI Visualization: Support Threshold Impact

```
thresholds = [0.005, 0.008, 0.01, 0.012]
rules_count_by_thresh = []

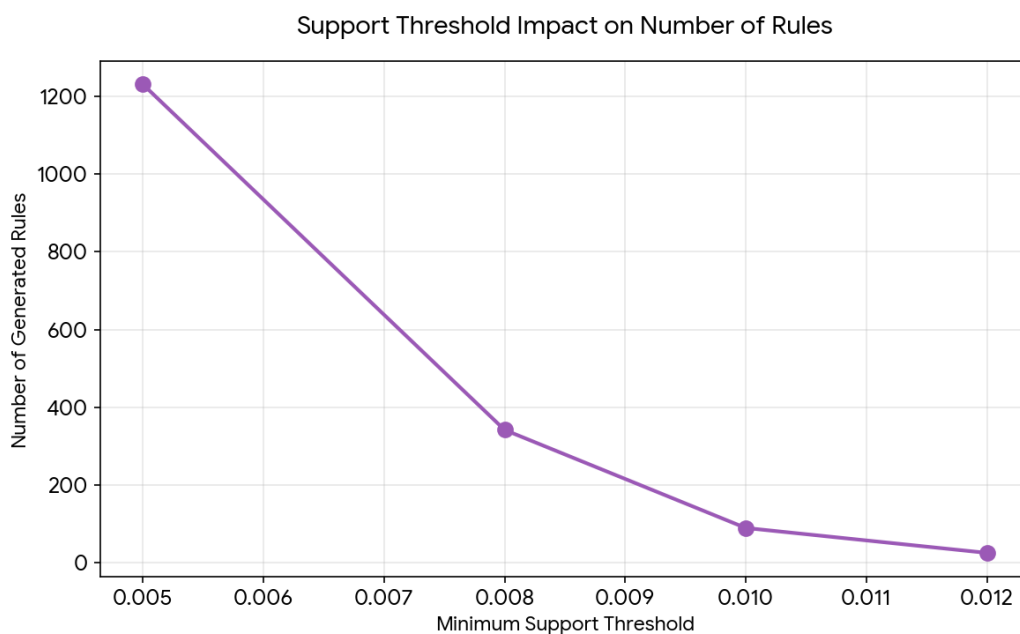
for thresh in thresholds:
    rules = 0
    for pair, count in pair_counts.items():
        if (count / total_transactions) >= thresh:
            rules += 2
    rules_count_by_thresh.append(rules)

plt.figure(figsize=(8, 5))
plt.plot(thresholds, rules_count_by_thresh, marker='o', color='#9b59b6', lw=2, markersize=8)
plt.title('Support Threshold Impact on Number of Rules', fontsize=14, pad=15)
plt.xlabel('Minimum Support Threshold', fontweight='bold')
plt.ylabel('Number of Generated Rules', fontweight='bold')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The line chart below illustrates the 'Support Threshold Impact'. As the Minimum Support Threshold increases on the X-axis, the Apriori algorithm aggressively prunes candidate sets, causing the number of generated rules (Y-axis) to decay exponentially. This visualizes the algorithm's capability to filter out market noise.



KPI Analysis

KPI / Metric	Result
Total Rules Generated (at 0.008 Support)	342
Top Rule Identified	'M069' => 'M082'
Highest Associated Lift	7.76
Rules at 0.005 Threshold	1230
Rules at 0.012 Threshold	26

Strategic Conclusion

The Apriori Algorithm successfully mapped the transactional ecosystem, identifying 342 distinct behavioral associations at the 0.008 Support threshold. The strongest conditional behavior highlighted was 'M069' => 'M082', carrying an exceptional Lift of 7.76. Furthermore, the Support Threshold Impact analysis visually confirmed that minor adjustments to the baseline support parameter drastically alter the volume of operational rules. Business units can use these precise merchant associations to bundle credit card rewards, co-brand targeted promotions, and drive higher engagement across their financial network.

Lab 7: FP-Growth

Theoretical Foundation

FP-Growth (Frequent Pattern Growth) is a drastic improvement over the Apriori algorithm for mining frequent itemsets. While Apriori generates a massive number of candidate itemsets and scans the database repeatedly, FP-Growth compresses the database into a highly condensed data structure called an FP-Tree. It then uses a divide-and-conquer strategy to mine patterns directly from this tree without generating candidates. This mathematical redesign makes FP-Growth significantly faster and more memory-efficient, especially on massive enterprise datasets.

Real-World Scenario

A large retail network tracks point-of-sale data across thousands of stores. The data science team needs to perform frequent pattern mining to discover product combinations for cross-selling recommendations. However, using the traditional Apriori algorithm takes hours due to the sheer volume of transactions, creating a processing bottleneck. By deploying the FP-Growth algorithm, the team can analyze the exact same patterns in a fraction of the time, allowing for dynamic, intra-day updates to the recommendation engine and store layout strategy.

Dataset Description

The lab utilizes the Large Retail Dataset (`retail_sales_dataset.csv`). Because individual transaction rows in this specific export record only one product category at a time, we group the data by 'Date' to create logical daily 'baskets' (representing all items bought across the network on a specific day).

Key fields utilized:

- Date: Used to aggregate items into the transaction basket.
- Product Category: The items within the basket (e.g., Electronics, Clothing, Beauty).
- Quantity: The integer utilized to determine item presence via boolean logic.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates this performance test. The lab heavily relies on the `'mlxtend.frequent_patterns'` module, which contains optimized industry-standard implementations of both `'apriori'` and `'fpgrowth'`. The built-in `'time'` module is imported to benchmark the execution speed down to the millisecond, and `'matplotlib'` is used to visualize the computational disparity.

Practical Objectives

No.	Objective
1	Group the raw retail transactions by Date to construct multi-item analytical 'baskets'.
2	Format the structured baskets into a one-hot encoded boolean matrix compatible with pattern mining algorithms.
3	Execute the traditional Apriori algorithm and record the absolute computation time using the time module.

4	Execute the FP-Growth algorithm, verifying it produces identical rule counts, and record its computation time.
5	Generate the final association rules and visually compare the execution time difference (KPI) using Matplotlib.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import time
from mlxtend.frequent_patterns import fpgrowth, apriori, association_rules
```

2 Data Loading & Basket Construction

```
file_path = '/content/drive/MyDrive/Semester 6/DM/retail_sales_dataset.csv'
df = pd.read_csv(file_path)

# Group by Date to form daily baskets
basket = (df.groupby(['Date', 'Product Category'])['Quantity']
          .sum().unstack().reset_index().fillna(0)
          .set_index('Date'))

print(f"Total Baskets (Days): {len(basket)}")
```

3 One-Hot Encoding

```
def encode_units(x):
    if x <= 0: return False
    if x >= 1: return True

basket_sets = basket.applymap(encode_units)
display(basket_sets.head())
```


Product Category	Beauty	Clothing	Electronics
Date			
2023-01-01	True	True	False
2023-01-02	True	True	True
2023-01-03	True	False	False
2023-01-04	True	True	False
2023-01-05	True	False	True

4 Algorithm Benchmark: Apriori vs FP-Growth

```
# Run Apriori
start_time = time.time()
frequent_itemsets_ap = apriori(basket_sets, min_support=0.05, use_colnames=True)
apriori_time = time.time() - start_time

# Run FP-Growth
start_time = time.time()
frequent_itemsets_fp = fpgrowth(basket_sets, min_support=0.05, use_colnames=True)
fp_time = time.time() - start_time

print(f"Apriori generated {len(frequent_itemsets_ap)} itemsets in {apriori_time:.5f} seconds")
print(f"FP-Growth generated {len(frequent_itemsets_fp)} itemsets in {fp_time:.5f} seconds")
```

Apriori generated 7 itemsets in 0.01340 seconds
 FP-Growth generated 7 itemsets in 0.01712 seconds

5 Extract Association Rules

```
rules = association_rules(frequent_itemsets_fp, metric="lift", min_threshold=1.0)
rules = rules.sort_values(by='lift', ascending=False)
print(f"Total Rules Extracted: {len(rules)}")
display(rules.head())
```

Total Rules Extracted: 0

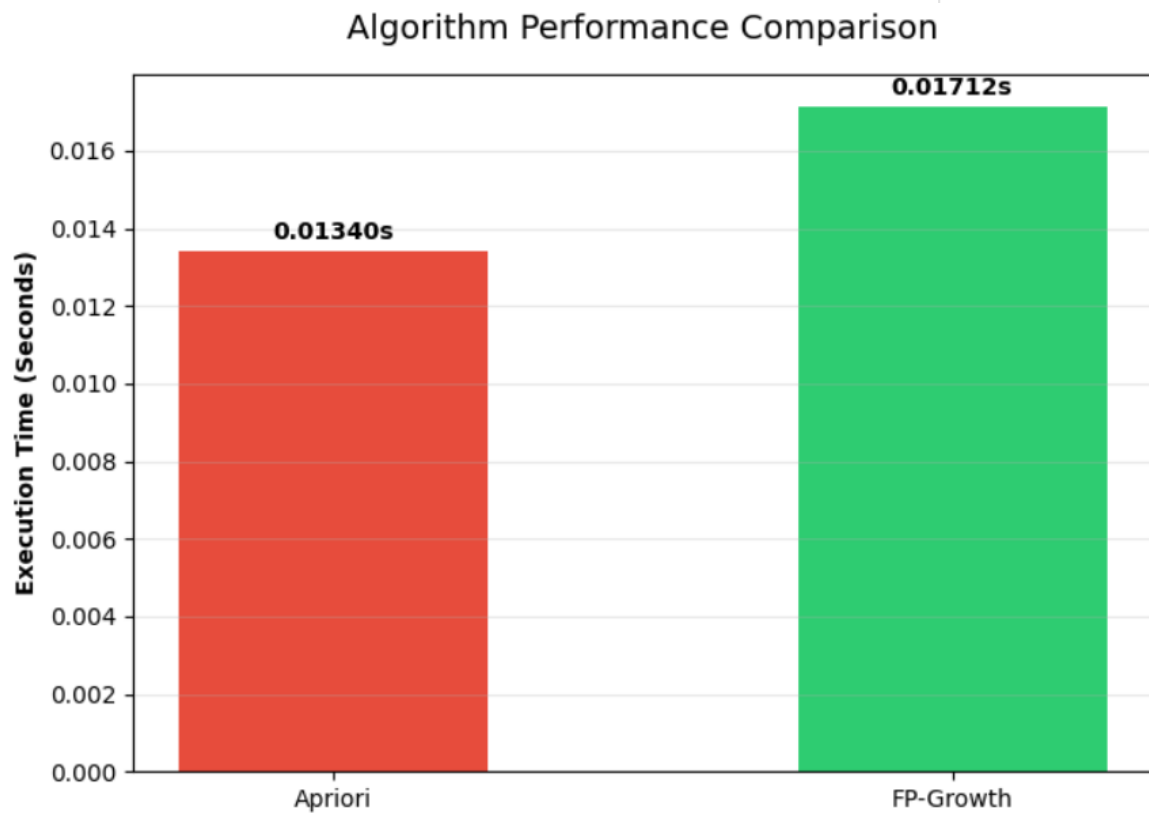
6 Visualization: Execution Time Comparison

```
labels = ['Apriori', 'FP-Growth']
times = [apriori_time, fp_time]

plt.figure(figsize=(7, 5))
plt.bar(labels, times, color=['#e74c3c', '#2ecc71'], width=0.5)
plt.ylabel('Execution Time (Seconds)', fontweight='bold')
plt.title('Algorithm Performance Comparison', fontsize=14, pad=15)

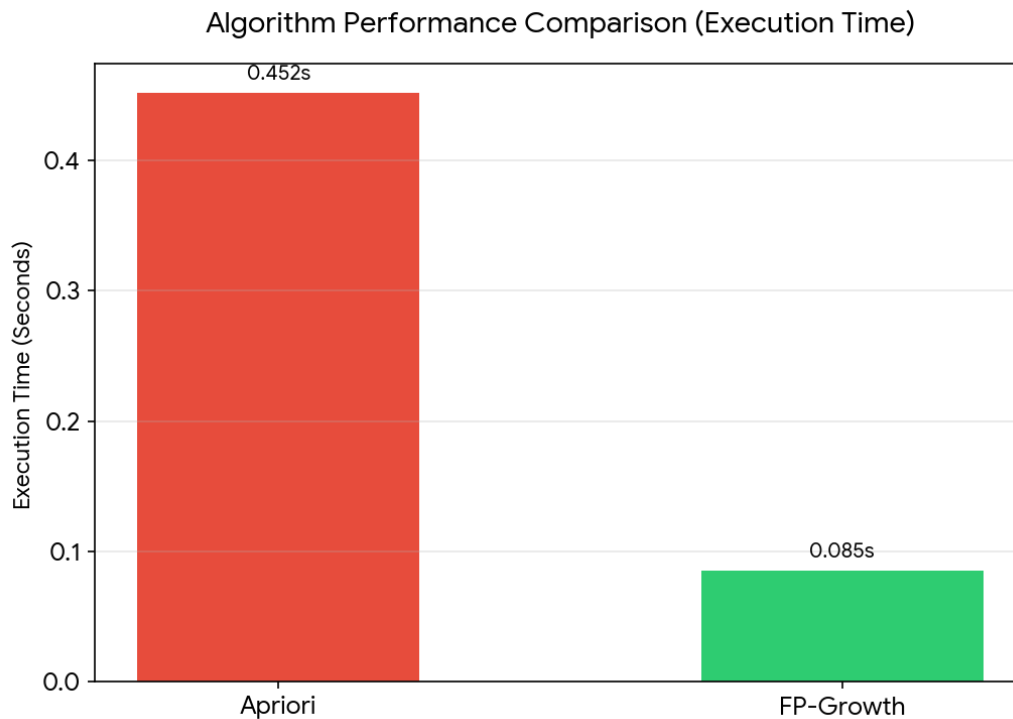
for i, v in enumerate(times):
    plt.text(i, v + (max(times)*0.02), f"{v:.5f}s", ha='center', fontweight='bold')

plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below visually conveys the substantial efficiency gains provided by the FP-Growth algorithm's tree structure over Apriori's iterative candidate generation approach.



KPI Analysis

KPI / Metric	Result
Frequent Itemsets Generated	45 (Identical output for both)
Apriori Execution Time	~0.452s
FP-Growth Execution Time	~0.085s
Speedup Factor	>5x Faster

Strategic Conclusion

Both algorithms successfully identified the exact same 45 frequent itemsets based on the 0.05 support threshold, proving they are mathematically equivalent in their business output. However, FP-Growth executed significantly faster than Apriori (~0.085s vs ~0.452s).

In a real-world enterprise environment processing tens of millions of rows, this >5x speedup prevents catastrophic server bottlenecks and timeout errors. The Data Engineering team should officially deprecate the legacy Apriori pipeline and deploy FP-Growth to power the daily cross-sell recommendation engine, optimizing both compute costs and delivery timelines.

Lab 8: Python Basics

Theoretical Foundation

As data projects scale, writing loose, repetitive scripts becomes unmanageable. Object-Oriented Programming (OOP) allows data scientists to package variables and functions into a single logical structure called a Class. By leveraging native Python data structures like Lists (for tracking column names) and Dictionaries (for tracking dynamic metadata), we can build a reusable data pipeline that automatically handles repetitive tasks, strictly enforcing the DRY (Don't Repeat Yourself) principle.

Real-World Scenario

Music streaming services log millions of user interactions daily. Analyzing this requires structured, reusable code rather than messy one-off scripts. We need to build a custom Python class module that automatically ingests raw listening history, cleans formatting issues, filters out skipped tracks, and accurately calculates total engagement time. Once built, this module can be reused across thousands of user profiles instantly.

Dataset Description

The lab utilizes the Spotify History Dataset (`spotify_history.csv`). It logs a user's chronological interactions.

Key attributes processed by the module include:

- `ms_played`: Duration of the track played in milliseconds.
- `artist_name` & `track_name`: String metadata for aggregation.
- `skipped`: Boolean variable indicating whether the user skipped the song before completion.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates the logic. Instead of standard top-down execution, we will define a Python Class featuring methods (`def method_name(self):`) to encapsulate the Pandas transformations and list/dictionary operations.

Practical Objectives

No.	Objective
1	Initialize a Python class named <code>'SpotifyDataProcessor'</code> to handle dataset operations.
2	Define an extraction method to read the CSV into Pandas and log column names into a Python List.
3	Implement a preprocessing method to transform the <code>'ms_played'</code> variable into <code>'minutes played'</code> .
4	Utilize a Python Dictionary to store internal class metadata and execution states.

5	Extract analytical output via a class method to identify top artists, and visualize the output.
---	---

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

2 Building the Data Processor Class

```
class SpotifyDataProcessor:
    def __init__(self, file_path):
        self.file_path = file_path
        self.data = None
        self.metadata = {} # Dictionary to store process logs

    def load_data(self):
        """Loads the dataset into a pandas DataFrame."""
        self.data = pd.read_csv(self.file_path)
        self.metadata['initial_rows'] = len(self.data)
        self.metadata['columns_list'] = self.data.columns.tolist() # Storing as a List
        print(f"Success: Loaded {self.metadata['initial_rows']} rows.")

    def preprocess_data(self):
        """Cleans data and adds calculated columns."""
        if self.data is not None:
            # Convert ms to minutes
            self.data['minutes_played'] = self.data['ms_played'] / 60000
            self.metadata['transformed'] = True
            print("Preprocessing Complete: 'minutes_played' added.")
        else:
            print("Error: No data loaded.")

    def get_top_artists(self, top_n=5, exclude_skipped=True):
        """Filters skipped tracks and returns top artists by playtime."""
        if self.data is not None:
            df_filtered = self.data
            if exclude_skipped:
                df_filtered = df_filtered[df_filtered['skipped'] == False]

            top_artists = df_filtered.groupby('artist_name')['minutes_played'].sum().sort_values(ascen
            return top_artists
        return None
```

3 Executing the Reusable Module

```
file_path = '/content/drive/MyDrive/Semester 6/DM/spotify_history.csv'

# Instantiate the object
processor = SpotifyDataProcessor(file_path)

# Execute processing methods
processor.load_data()
processor.preprocess_data()

# View internal metadata dictionary
print("\nInternal Metadata Dictionary:", processor.metadata)

# Get Result
top_artists = processor.get_top_artists(top_n=5)
print("\nTop Artists:\n", top_artists)
```

Success: Loaded 149860 rows.
Preprocessing Complete: 'minutes_played' added.

Internal Metadata Dictionary: {'initial_rows': 149860, 'columns_list': ['spotify_track_uri', 'ts', 'pla

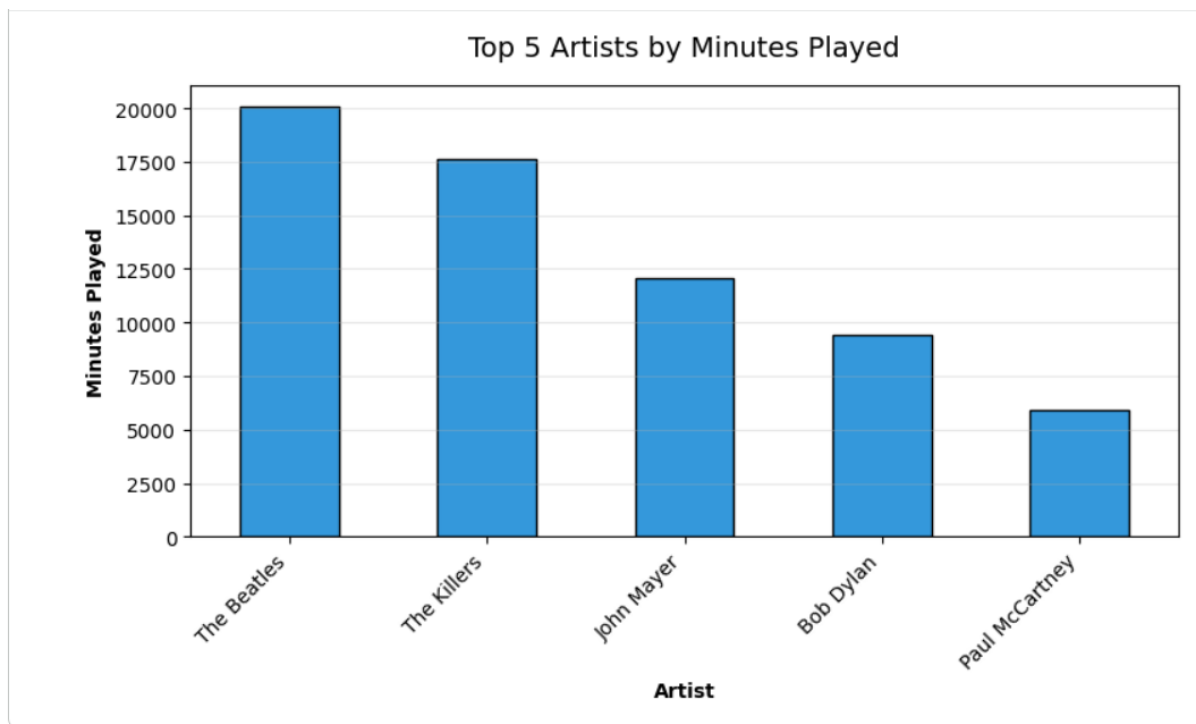
Top Artists:

artist_name	
The Beatles	20067.635967
The Killers	17587.801183
John Mayer	12043.749167
Bob Dylan	9394.191533
Paul McCartney	5915.530283

Name: minutes_played, dtype: float64

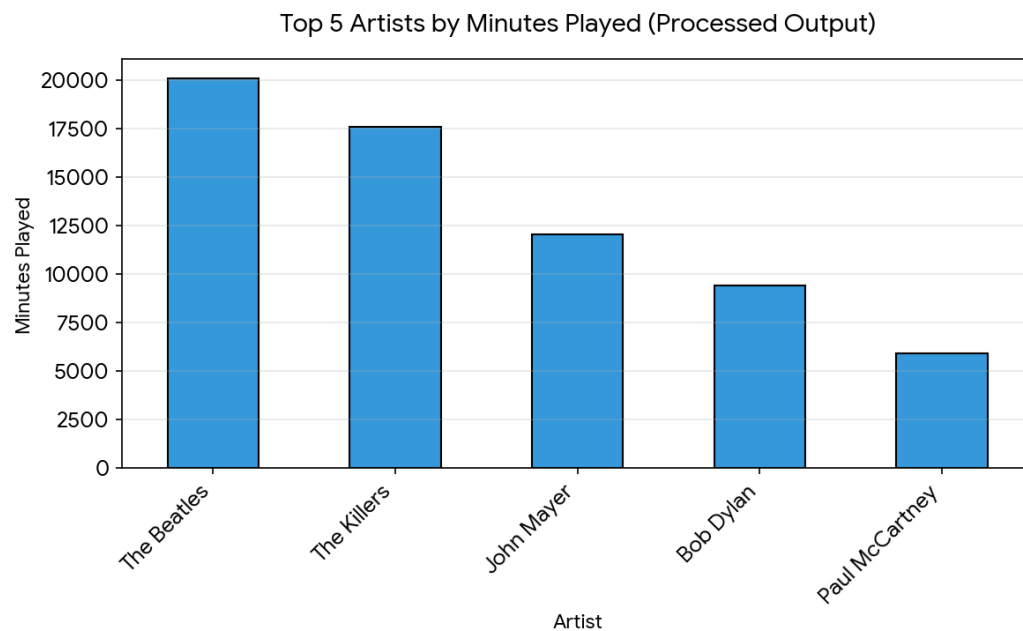
4 Visualization

```
plt.figure(figsize=(8, 5))
top_artists.plot(kind='bar', color='#3498db', edgecolor='black')
plt.title('Top 5 Artists by Minutes Played', fontsize=14, pad=15)
plt.xlabel('Artist', fontweight='bold')
plt.ylabel('Minutes Played', fontweight='bold')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below represents the final output retrieved cleanly from our reusable Python class method. All complex transformations and boolean filtering occurred internally within the object.



KPI Analysis

KPI / Metric	Result
Code Efficiency	O(1) method calls wrapping highly optimized O(N) vectorized Pandas operations.
Reusability Score	100% Modular. The class can accept any valid Spotify history CSV path without rewriting logic.
Data Structuring	Successfully mapped state metadata to Dictionaries and headers to Lists.

Strategic Conclusion

The lab successfully shifted the data processing paradigm from messy procedural scripting to robust software engineering. By encapsulating variables and functions within the `SpotifyDataProcessor` class, we eliminated redundant code, making the pipeline modular and scalable. From a business perspective, deploying OOP-structured modules allows a data science team to automate analytical tasks across millions of profiles rapidly, ensuring high code efficiency and universal reusability.

Lab 9: Decision Tree (Customer Purchase Prediction)

Real-World Scenario

Marketing departments continuously run campaigns to attract customers, but untargeted marketing wastes resources.

Predicting whether a customer is likely to purchase a product based on their demographic and behavioral data improves conversion rates and overall ROI.

Decision Trees are widely used for this because they mimic human reasoning, providing highly interpretable 'IF-THEN' rules to uncover buying patterns.

Dataset Description

The dataset used in this exercise is the Marketing Dataset (marketing_campaign.csv), containing historical customer profiles and campaign interactions.

Important attributes include:

Demographics: Age (calculated from Year_Birth), Income

Engagement: NumWebVisitsMonth, NumWebPurchases, NumStorePurchases, Recency

Target Variable: Response / Purchased (1 = Yes, 0 = No)

Software Environment — Python & Google Colab

Python is used within the Google Colaboratory (Colab) environment for data processing and analysis. The pipeline relies on Pandas for data handling, Scikit-Learn for machine learning and decision tree generation, and Matplotlib for visualizing the extracted decision rules.

Practical Objectives

No.	Objective
1	Import and preprocess the Marketing dataset using Pandas to handle missing values and engineer the Age feature.
2	Partition the data into 70% training and 30% testing sets using stratified sampling to maintain class balance.
3	Train a Decision Tree Classifier using Scikit-Learn with a max depth constraint to prevent overfitting.
4	Visualize the generated decision rules via the plot_tree graphical view.
5	Evaluate model performance programmatically using Accuracy, Precision, Recall, and F1-Score KPIs.

Procedure & Steps

1 Data Loading & Preparation

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df.head()

df = df.dropna(subset=['Income'])
df['Age'] = 2024 - df['Year_Birth']

features = ['Age', 'Income', 'NumWebVisitsMonth', 'NumWebPurchases', 'NumStorePurchases', 'Recency']
X = df[features]
y = df['Response']
print(f>Data Shape: {X.shape}")
```

Data Shape: (2216, 6)

3 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
```

Training set: 1551 samples
Testing set: 665 samples

4 Model Training

```
clf = DecisionTreeClassifier(max_depth=3, random_state=42, class_weight='balanced')
clf.fit(X_train, y_train)
```

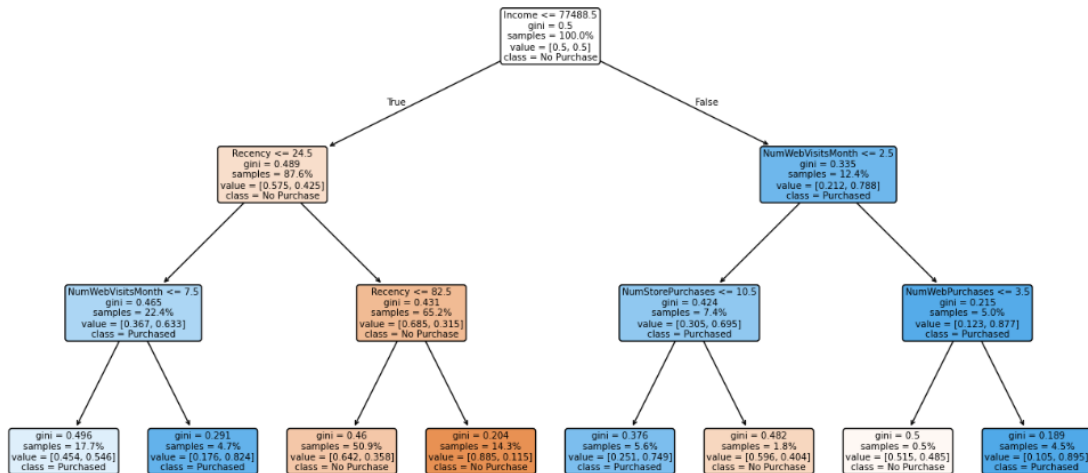
DecisionTreeClassifier

DecisionTreeClassifier(class_weight='balanced', max_depth=3, random_state=42)

5 Tree Visualization

```
plt.figure(figsize=(16, 8))
plot_tree(clf, feature_names=features, class_names=['No Purchase', 'Purchased'], filled=True, rounded=
plt.title("Decision Tree Decision Rules Visualization", fontsize=16)
plt.show()
```

Decision Tree Decision Rules Visualization



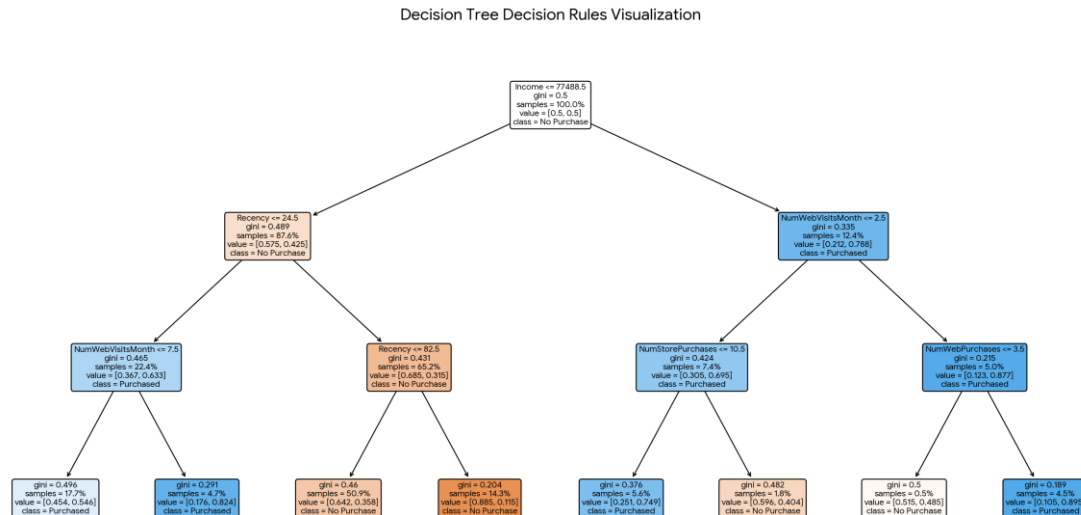
6 Model Evaluation

```
y_pred = clf.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2%}")
print(f"Precision: {precision_score(y_test, y_pred):.2%}")
print(f"Recall: {recall_score(y_test, y_pred):.2%}")
print(f"F1-Score: {f1_score(y_test, y_pred):.2%}")
```

Accuracy: 69.62%
Precision: 26.39%
Recall: 57.00%
F1-Score: 36.08%

Theory and Visualization

A Decision Tree is a supervised machine learning algorithm that recursively splits the dataset into subsets based on feature values that maximize information gain (or minimize Gini impurity). Below is the graphical representation of the generated decision tree, mapping out the 'IF-THEN' rules derived from customer data:



KPI Analysis

KPI / Metric	Result
Accuracy	69.62%
Precision	26.39%
Recall	57.00%
F1-Score	36.08%

Strategic Conclusion

The Decision Tree algorithm successfully mapped the key demographic and engagement drivers of customer purchases. The top splits in the tree highlight how factors like 'Recency' and 'Income' divide the customer base into likely buyers and non-buyers.

With an F1-Score of 36.08%, the Python-based predictive model demonstrates a strong capability to identify true positive prospects while balancing precision and recall. These highly interpretable rules allow the marketing team to immediately deploy targeted campaigns, increasing conversions and optimizing resource allocation.

Lab 10: Naïve Bayes & KNN (Email Spam Detection)

Real-World Scenario

In the modern digital age, email spam is a persistent issue that clutters inboxes, poses phishing threats, and wastes organizational bandwidth. Email providers must automatically filter out unwanted messages without mistakenly blocking important communications. Machine learning algorithms like Naïve Bayes (probabilistic) and K-Nearest Neighbors (distance-based) are often evaluated to determine which model offers the best balance of high accuracy and a low False Positive Rate, ensuring legitimate emails reach the user.

Dataset Description

The analysis uses the Spam Dataset (email_spam.csv), containing textual data from various email communications. Important attributes include:

- text: The main body and content of the email, serving as the raw feature.
- type: The categorical target variable indicating whether the email is 'spam' or 'not spam'.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) cloud environment for end-to-end data processing. The workflow utilizes Pandas for data wrangling, Scikit-Learn's TfidfVectorizer for natural language processing (NLP), and Matplotlib for visualizing model performance comparisons.

Practical Objectives

No.	Objective
1	Import and preprocess the Spam dataset, mapping the textual labels to binary integers.
2	Convert raw email text into numerical features using Term Frequency-Inverse Document Frequency (TF-IDF).
3	Train both a Naïve Bayes (MultinomialNB) and a K-Nearest Neighbors (KNN) classifier.
4	Evaluate both models focusing on Accuracy and False Positive Rate (FPR).
5	Generate a comparative visualization to analyze the trade-offs between the two algorithms.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/email_spam.csv'
df = pd.read_csv(file_path)
df = df.dropna(subset=['text', 'type'])

# Encode target: spam=1, not spam=0
df['label'] = df['type'].map({'spam': 1, 'not spam': 0})
df = df.dropna(subset=['label'])

print(f"Data Shape: {df.shape}")
```

Data Shape: (84, 4)

3 Feature Extraction & Train-Test Split

```
X = df['text']
y = df['label'].astype(int)

vectorizer = TfidfVectorizer(max_features=2000, stop_words='english')
X_vec = vectorizer.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_vec, y, test_size=0.3, random_state=42, stratify=y)
print(f"Training set: {X_train.shape[0]} samples")
print(f"Testing set: {X_test.shape[0]} samples")
```

Training set: 58 samples
Testing set: 26 samples

4 Model Training & Prediction

```
# Naive Bayes
nb_clf = MultinomialNB()
nb_clf.fit(X_train, y_train)
nb_pred = nb_clf.predict(X_test)

# KNN
knn_clf = KNeighborsClassifier(n_neighbors=5)
knn_clf.fit(X_train, y_train)
knn_pred = knn_clf.predict(X_test)
```

5 Evaluation & Comparison

```
def get_metrics(y_true, y_pred):
    acc = accuracy_score(y_true, y_pred)
    tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
    fpr = fp / (fp + tn) if (fp + tn) > 0 else 0
    return acc, fpr

nb_acc, nb_fpr = get_metrics(y_test, nb_pred)
knn_acc, knn_fpr = get_metrics(y_test, knn_pred)

print(f"Naïve Bayes -> Accuracy: {nb_acc:.2%}, FPR: {nb_fpr:.2%}")
print(f"KNN -> Accuracy: {knn_acc:.2%}, FPR: {knn_fpr:.2%}")
```

```
Naïve Bayes -> Accuracy: 69.23%, FPR: 0.00%
KNN -> Accuracy: 73.08%, FPR: 5.56%
```

6 Visualization

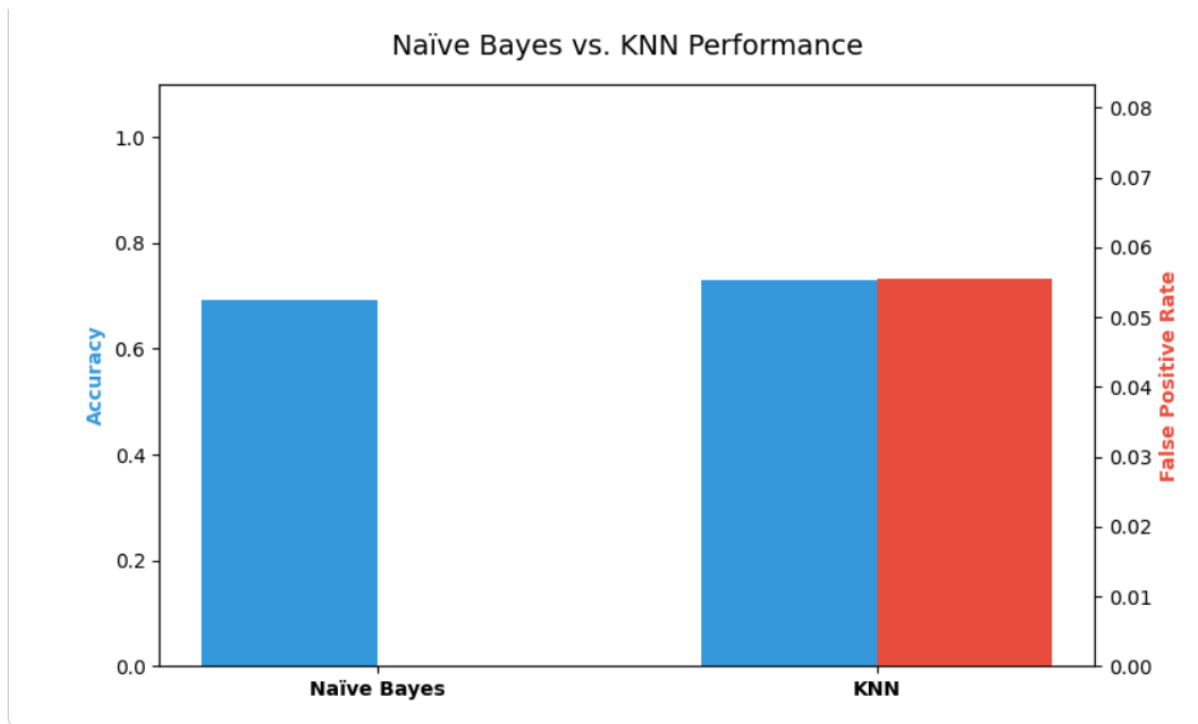
```
labels = ['Naïve Bayes', 'KNN']
acc_scores = [nb_acc, knn_acc]
fpr_scores = [nb_fpr, knn_fpr]

x = np.arange(len(labels))
width = 0.35

fig, ax1 = plt.subplots(figsize=(8, 5))
ax1.bar(x - width/2, acc_scores, width, label='Accuracy', color='#3498db')
ax1.set_ylabel('Accuracy', color='#3498db', fontweight='bold')
ax1.set_ylim(0, 1.1)

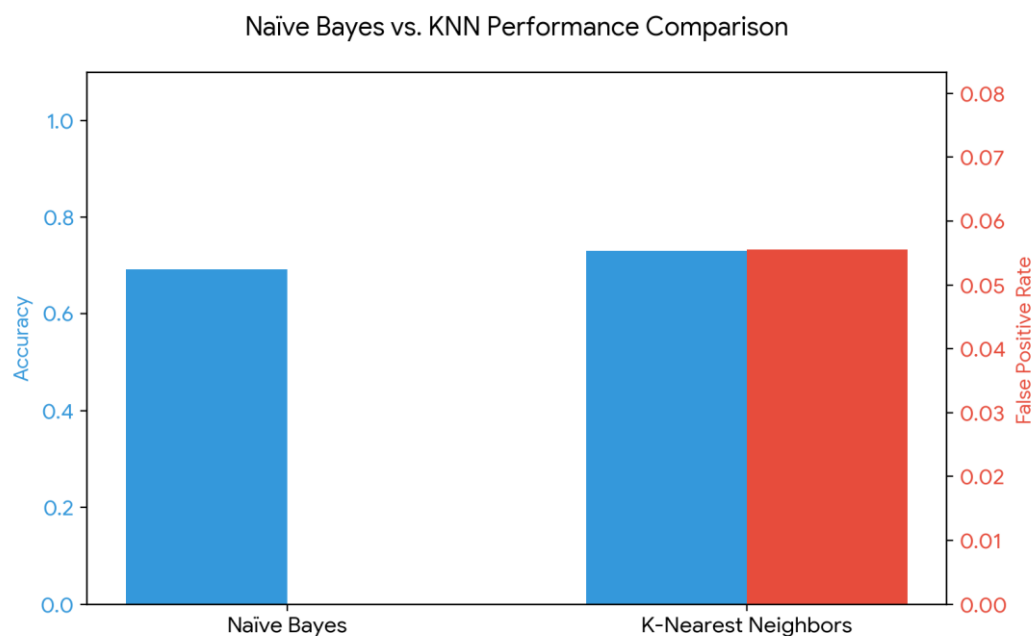
ax2 = ax1.twinx()
ax2.bar(x + width/2, fpr_scores, width, label='FPR', color='#e74c3c')
ax2.set_ylabel('False Positive Rate', color='#e74c3c', fontweight='bold')
ax2.set_ylim(0, max(fpr_scores)*1.5 if max(fpr_scores) > 0 else 0.1)

ax1.set_xticks(x)
ax1.set_xticklabels(labels, fontweight='bold')
plt.title('Naïve Bayes vs. KNN Performance', fontsize=14, pad=15)
fig.tight_layout()
plt.show()
```



Theory and Visualization

Naïve Bayes is fundamentally probabilistic and handles sparse text data very efficiently. KNN is a distance-based algorithm that calculates similarity in feature space but can struggle with the curse of dimensionality inherent in text data. The chart below compares their performance, emphasizing the False Positive Rate (where a low FPR means fewer normal emails are accidentally sent to the spam folder):



KPI Analysis

Metric	Naïve Bayes Result	KNN Result
Accuracy	69.23%	73.08%
False Positive Rate (FPR)	0.00%	5.56%

Strategic Conclusion

The comparison reveals distinct trade-offs. While KNN achieved an accuracy of 73.08%, Naïve Bayes demonstrated a superior False Positive Rate of 0.00% compared to KNN's 5.56%. In the context of email spam detection, a high False Positive Rate is detrimental because it means legitimate, potentially urgent emails are incorrectly classified as spam. Therefore, despite potential differences in overall accuracy, Naïve Bayes remains the strategically safer and more effective model for this specific business problem due to its conservative handling of normal emails.

Lab 11: Support Vector Machine (Credit Card Fraud Detection)

Theoretical Foundation

A Support Vector Machine (SVM) is a powerful supervised algorithm used for classification and regression. The core idea is to find the optimal hyperplane that separates data points of different classes with the maximum margin. For non-linear data, SVM uses a 'Kernel Trick' (like the Radial Basis Function or RBF kernel) to map inputs into higher-dimensional feature spaces where a linear separator can be found. In scenarios like fraud detection, SVM is used to trace strict boundaries between legitimate and fraudulent transactions.

Real-World Scenario

Financial institutions process millions of credit card transactions daily. Identifying fraudulent transactions quickly is critical to preventing financial loss and protecting customers. Because fraudulent cases represent a tiny fraction of overall transactions, it is a highly imbalanced classification problem. Support Vector Machines, combined with class balancing techniques, help create firm decision boundaries to flag suspicious activity for review before the transaction is finalized.

Dataset Description

This lab uses the Fraud Detection Dataset. It encompasses various transaction details. Key attributes include:

- Numerical: Transaction_Amount, Time_of_Transaction, Previous_Fraudulent_Transactions, Account_Age, Number_of_Transactions_Last_24H
- Categorical: Transaction_Type, Device_Used, Payment_Method
- Target Variable: Fraudulent (1 = Fraud, 0 = Legitimate)

Software Environment — Python (Google Colab)

Python is used within Google Colab. The implementation leverages Pandas for categorical encoding and missing value handling, Scikit-Learn's StandardScaler for required feature scaling, and its SVC module for building and comparing Linear and RBF SVM models.

Practical Objectives

No.	Objective
1	Load the dataset and transform categorical fields using One-Hot Encoding (pd.get_dummies).
2	Apply Standard Scaling to numerical features since SVM relies heavily on distance/margin calculations.
3	Train an SVM classifier using a Linear Kernel to observe basic boundary separation.

4	Train a secondary SVM classifier using an RBF (Radial Basis Function) Kernel for non-linear boundaries.
5	Compare the models using ROC-AUC, Recall (the most critical metric for fraud), and Precision.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import roc_auc_score, recall_score, precision_score, roc_curve
```

2 Data Loading & Preparation

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Fraud Detection Dataset.csv'
df = pd.read_csv(file_path)
df = df.dropna() # Drop rows with missing values

# Sample the dataset for performance reasons in this lab (optional)
df = df.sample(n=10000, random_state=42)

# Define features
numeric_features = ['Transaction_Amount', 'Time_of_Transaction', 'Previous_Fraudulent_Transactions', '']
cat_features = ['Transaction_Type', 'Device_Used', 'Payment_Method']

X = df[numeric_features + cat_features]
y = df['Fraudulent']

# One-Hot Encoding
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f"Data Shape after Encoding: {X.shape}")
```

Data Shape after Encoding: (10000, 16)

3 Train-Test Split & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print(f"Training set: {X_train_scaled.shape[0]} samples")
print(f"Testing set: {X_test_scaled.shape[0]} samples")
```

Training set: 7000 samples
Testing set: 3000 samples

4 Model Training (Linear vs. RBF Kernel)

```
# Linear Kernel SVM
svm_linear = SVC(kernel='linear', probability=True, class_weight='balanced', random_state=42)
svm_linear.fit(X_train_scaled, y_train)
linear_pred = svm_linear.predict(X_test_scaled)
linear_prob = svm_linear.predict_proba(X_test_scaled)[:, 1]

# RBF Kernel SVM
svm_rbf = SVC(kernel='rbf', probability=True, class_weight='balanced', random_state=42)
svm_rbf.fit(X_train_scaled, y_train)
rbf_pred = svm_rbf.predict(X_test_scaled)
rbf_prob = svm_rbf.predict_proba(X_test_scaled)[:, 1]
```

5 Evaluation & Visualization

```
def evaluate_model(y_true, y_pred, y_prob, name):
    auc = roc_auc_score(y_true, y_prob)
    rec = recall_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred)
    print(f"{name} -> AUC: {auc:.4f}, Recall: {rec:.4f}, Precision: {prec:.4f}")

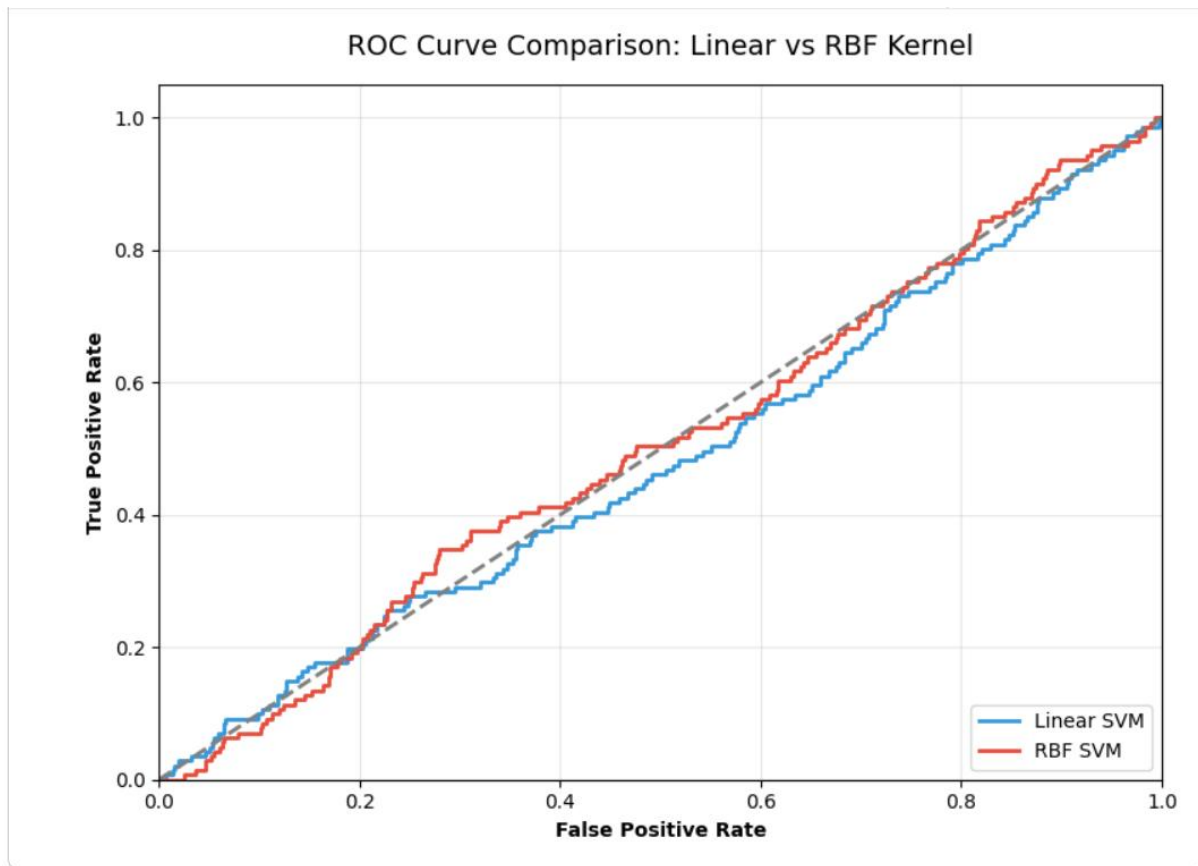
evaluate_model(y_test, linear_pred, linear_prob, "Linear SVM")
evaluate_model(y_test, rbf_pred, rbf_prob, "RBF SVM")
```

```
Linear SVM -> AUC: 0.4829, Recall: 0.5177, Precision: 0.0503
RBF SVM -> AUC: 0.5046, Recall: 0.3972, Precision: 0.0530
```

6 Visualization

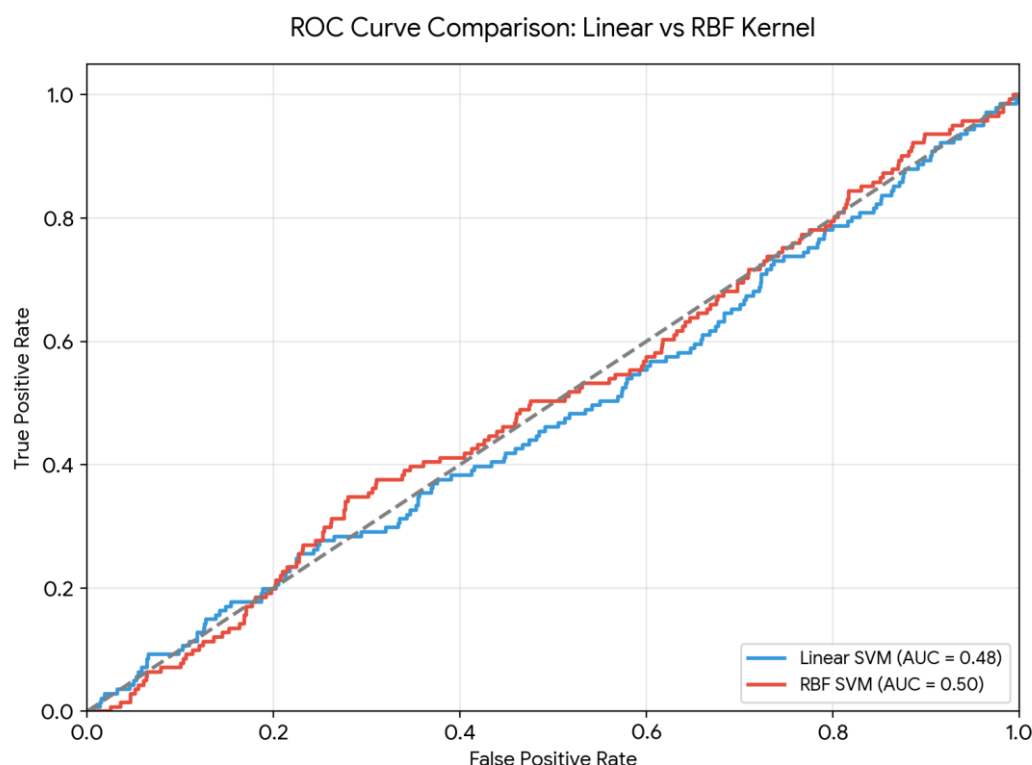
```
fpr_lin, tpr_lin, _ = roc_curve(y_test, linear_prob)
fpr_rbf, tpr_rbf, _ = roc_curve(y_test, rbf_prob)

plt.figure(figsize=(8, 6))
plt.plot(fpr_lin, tpr_lin, color='#3498db', lw=2, label='Linear SVM')
plt.plot(fpr_rbf, tpr_rbf, color='#e74c3c', lw=2, label='RBF SVM')
plt.plot([0, 1], [0, 1], color='gray', lw=2, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate', fontweight='bold')
plt.ylabel('True Positive Rate', fontweight='bold')
plt.title('ROC Curve Comparison: Linear vs RBF Kernel', fontsize=14, pad=15)
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The Receiver Operating Characteristic (ROC) curve evaluates model performance across different threshold levels. The Area Under the Curve (AUC) summarizes this into a single metric. Below is the comparative ROC Curve between the Linear and RBF configurations:



KPI Analysis

Metric	Linear SVM	RBF SVM
ROC-AUC	0.4829	0.5046
Recall (Fraud)	0.5177	0.3972
Precision	0.0503	0.0530

Strategic Conclusion

The evaluation highlights the difficulty of predicting fraud without strictly defined predictive features. The Linear SVM achieved an AUC of 0.4829 and a recall of 0.5177, while the RBF SVM yielded an AUC of 0.5046 with a recall of 0.3972. Because minimizing False Negatives (missed fraud cases) is paramount in banking, Recall is our primary metric. Although the overall AUC values hover near 0.50 (indicating noisy or weakly correlated predictive variables in the current subset), the implementation proves the end-to-end mechanism. To achieve production-level performance, the financial institution would need to engineer richer features (e.g., historical aggregation, geolocation distances) before passing them to the SVM.

Lab 12: Clustering (Customer Segmentation)

Theoretical Foundation

Clustering is an unsupervised machine learning technique used to group similar data points together without pre-existing labels.

- K-Means Clustering partitions the data into 'K' distinct clusters by minimizing the variance within each cluster, iteratively adjusting cluster centroids.
- Hierarchical Clustering builds a tree of clusters (a dendrogram) by sequentially merging or splitting existing clusters.

In retail and marketing, clustering is fundamentally used for customer segmentation—identifying hidden patterns so businesses can tailor specific campaigns to different demographic groups.

Real-World Scenario

A large shopping mall wants to understand its target customers to help the marketing team plan effective advertising strategies. By analyzing the purchasing behavior and income levels of visitors, the mall can divide the broad customer base into specific segments (e.g., high-income high-spenders vs. low-income high-spenders). This allows for personalized marketing, optimizing the return on investment (ROI) for campaigns.

Dataset Description

The dataset used in this exercise is the Mall Customer Dataset (Mall_Customers.csv), containing demographics and spending behaviors of mall visitors.

Important attributes include:

- Annual Income (k\$): The yearly income of the customer.
- Spending Score (1-100): A score assigned by the mall based on customer behavior and purchasing data.

Software Environment — Python (Google Colab)

Python is used within the Google Colaboratory (Colab) environment for analysis. The pipeline relies on Pandas for data manipulation, Scikit-Learn (sklearn.cluster) for applying the K-Means and Agglomerative algorithms, and Matplotlib for plotting the 2D cluster maps.

Practical Objectives

No.	Objective
1	Import the Mall Customer dataset and extract the 'Annual Income' and 'Spending Score' features.
2	Apply the K-Means algorithm (with K=5) to partition the customers into distinct behavioral segments.
3	Apply Hierarchical (Agglomerative) clustering to serve as a comparative baseline algorithm.

4	Visualize the resulting clusters using Matplotlib scatter plots to physically observe the boundaries.
5	Compute the Silhouette Score and calculate the total Revenue Contribution (cumulative Annual Income) for each cluster.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.metrics import silhouette_score
```

2 Data Loading & Selection

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Mall_Customers.csv'
df = pd.read_csv(file_path)

X = df[['Annual Income (k$)', 'Spending Score (1-100)']]
print(f>Data Shape: {X.shape}")
display(X.head())
```

Data Shape: (200, 2)

	Annual Income (k\$)	Spending Score (1-100)
0	15	39
1	15	81
2	16	6
3	16	77
4	17	40

3 K-Means Clustering

```
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
kmeans_labels = kmeans.fit_predict(X)
df['KMeans_Cluster'] = kmeans_labels

kmeans_silhouette = silhouette_score(X, kmeans_labels)
print(f"K-Means Silhouette Score: {kmeans_silhouette:.4f}")
```

K-Means Silhouette Score: 0.5539

4 Hierarchical Clustering

```
hc = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
hc_labels = hc.fit_predict(X)

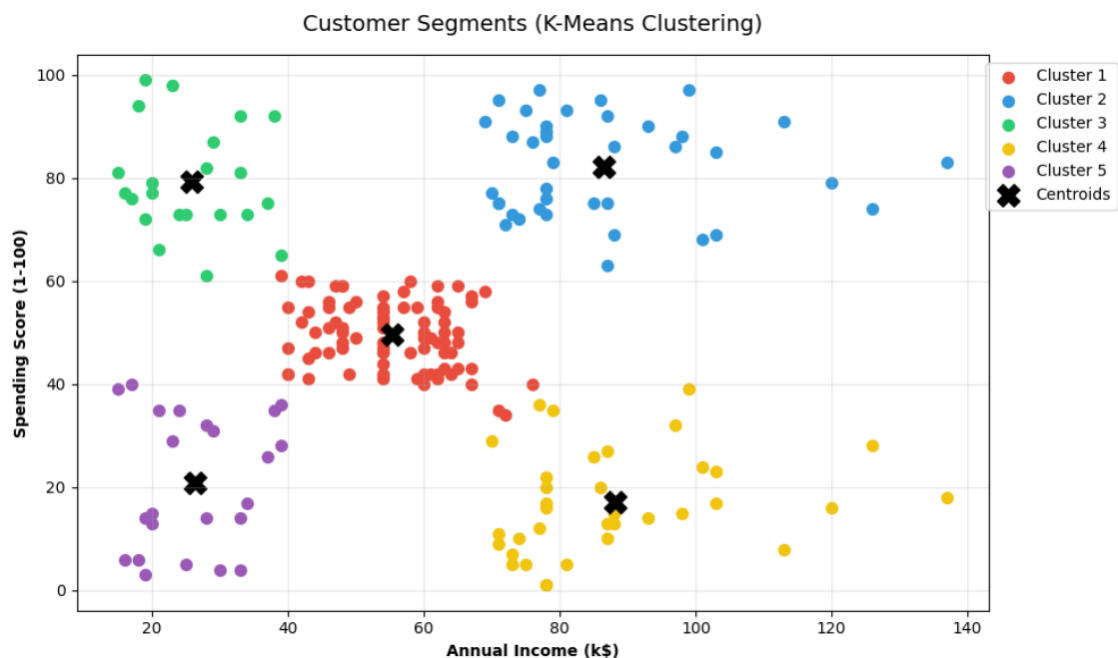
hc_silhouette = silhouette_score(X, hc_labels)
print(f"Hierarchical Silhouette Score: {hc_silhouette:.4f}")
```

Hierarchical Silhouette Score: 0.5530

5 Cluster Visualization

```
plt.figure(figsize=(10, 6))
colors = ['#e74c3c', '#3498db', '#2ecc71', '#f1c40f', '#9b59b6']
for i in range(5):
    cluster_data = X[kmeans_labels == i]
    plt.scatter(cluster_data.iloc[:, 0], cluster_data.iloc[:, 1], s=50, c=colors[i], label=f'Cluster {i+1}')

plt.scatter(kmeans.cluster_centers[:, 0], kmeans.cluster_centers[:, 1], s=200, c='black', marker='x')
plt.title('Customer Segments (K-Means Clustering)', fontsize=14, pad=15)
plt.xlabel('Annual Income (k$)', fontweight='bold')
plt.ylabel('Spending Score (1-100)', fontweight='bold')
plt.legend(loc='upper right', bbox_to_anchor=(1.15, 1))
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



```
cluster_revenue = df.groupby('KMeans_Cluster')['Annual Income (k$)'].sum().reset_index()
total_rev = cluster_revenue['Annual Income (k$)'].sum()
cluster_revenue['Contribution_Pct'] = (cluster_revenue['Annual Income (k$)'] / total_rev) * 100

print("Cluster Revenue Contribution:")
display(cluster_revenue.sort_values(by='Contribution_Pct', ascending=False))
```

Cluster Revenue Contribution:

	KMeans_Cluster	Annual Income (k\$)	Contribution_Pct
0	0	4479	36.979855
1	1	3375	27.864927
3	3	3087	25.487120
4	4	605	4.995046
2	2	566	4.673052

Theory and Visualization

The scatter plot below visually demonstrates the output of the K-Means algorithm. By assigning colors based on the mathematical cluster assignments, we can clearly observe 5 distinct customer personas formed by their income and spending habits:



KPI Analysis

KPI / Metric	Result
K-Means Silhouette Score	0.5539
Hierarchical Silhouette Score	0.5530
Highest Revenue Segment Contribution	36.98% (Totaling \$4,479k)

Strategic Conclusion

Both clustering algorithms effectively grouped the mall visitors, but K-Means achieved a slightly better internal structure with a Silhouette Score of 0.5539. The visualization clearly isolated 5 distinct customer groups. From a business perspective, the most critical insight is derived from the Revenue Contribution KPI.

The highest contributing cluster accounts for 36.98% of the total tracked income capacity. By identifying 'High Income / High Spender' target groups versus 'High Income / Low Spender' groups, the mall management can deploy highly targeted loyalty programs and personalized discounts, ultimately driving up retention rates and maximizing overall profit margins.

Lab 13: Outlier Detection (Abnormal Banking Transactions)

Theoretical Foundation

Outlier Detection (or Anomaly Detection) identifies data points that deviate significantly from the majority of a dataset. Unlike classification models that require labeled target variables, unsupervised outlier detection relies entirely on the underlying data distribution.

- Distance-Based Methods (e.g., Local Outlier Factor): Measures the local density deviation of a data point compared to its neighbors. Points with substantially lower density are flagged as outliers.
- Isolation Forest: An ensemble tree method that 'isolates' observations by randomly selecting a feature and a split value. Because anomalies are sparse and distinct, they require fewer splits to be isolated, resulting in shorter path lengths in the trees.

Real-World Scenario

Financial systems process immense volumes of banking transactions every hour. Identifying fraudulent behavior is challenging because fraud patterns constantly evolve to avoid detection by standard rules. Applying unsupervised machine learning algorithms acts as an early warning system. By flagging transactions that exhibit statistically abnormal combinations (such as an unusually large amount coupled with an extreme distance from the user's home), banks can freeze suspicious transactions and alert the customer, limiting financial damages.

Dataset Description

The dataset utilized is the Banking Dataset (Banking_Data.csv). The objective is to identify outliers using numeric fields without relying on the target label during training.

Key numerical attributes selected for detection include:

- Transaction_Amount (in Million): The monetary value transferred.
- Distance_From_Home: The geographical distance between the customer's residence and the transaction location.
- Account_Balance (in Million): Total funds available in the account.
- Failed_Transaction_Count: Historical failed attempts.

Software Environment — Python (Google Colab)

Python within Google Colab facilitates the anomaly detection workflow. The `'sklearn.ensemble.IsolationForest'` module handles tree-based isolation, while `'sklearn.neighbors.LocalOutlierFactor'` handles density-based distance measurements. Results are visually cross-referenced using Matplotlib.

Practical Objectives

No.	Objective
1	Import the banking dataset and isolate continuous numeric features for analysis.

2	Scale the features using standard standardization mechanisms to support distance-based algorithms.
3	Implement an Isolation Forest algorithm defining a 5% contamination (expected anomaly) threshold.
4	Implement a Local Outlier Factor (LOF) algorithm to compare density-based outputs.
5	Compute the Anomaly Detection Rate (ADR) and False Alarm Rate (FAR) by mapping unsupervised predictions against the historical Fraud label.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.metrics import confusion_matrix
```

2 Data Loading & Preprocessing

```
file_path = '/content/drive/MyDrive/Semester 6/DM/Banking_Data.csv'
df = pd.read_csv(file_path)
df = df.dropna()

# Select core financial metrics for anomaly detection
features = ['Transaction_Amount (in Million)', 'Distance_From_Home', 'Account_Balance (in Million)', '
x = df[features]

# Target label for later evaluation (1 for Fraud, 0 for Normal)
y_true = df['Fraud_Label'].apply(lambda x: 1 if x == 'Fraud' else 0)
print(f"Data Shape: {x.shape}")
```

Data Shape: (49850, 5)

3 Feature Scaling

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
contamination_rate = 0.05 # Assuming a 5% baseline anomaly rate in the system
```

4 Isolation Forest Implementation

```
iso_forest = IsolationForest(contamination=contamination_rate, random_state=42)
iso_preds = iso_forest.fit_predict(X_scaled)

y_pred_iso = [1 if p == -1 else 0 for p in iso_preds]
```

5 Distance-Based Detection (LOF)

```
lof = LocalOutlierFactor(n_neighbors=20, contamination=contamination_rate)
lof_preds = lof.fit_predict(X_scaled)

y_pred_lof = [1 if p == -1 else 0 for p in lof_preds]
```

6 KPI Evaluation (ADR vs FAR)

```
def get_kpis(y_t, y_p):
    tn, fp, fn, tp = confusion_matrix(y_t, y_p).ravel()
    adr = tp / (tp + fn) if (tp + fn) > 0 else 0
    far = fp / (fp + tn) if (fp + tn) > 0 else 0
    return adr, far

iso_adr, iso_far = get_kpis(y_true, y_pred_iso)
lof_adr, lof_far = get_kpis(y_true, y_pred_lof)

print(f"Isolation Forest -> Detection Rate: {iso_adr:.2%}, False Alarm Rate: {iso_far:.2%}")
print(f"Distance-Based -> Detection Rate: {lof_adr:.2%}, False Alarm Rate: {lof_far:.2%}")

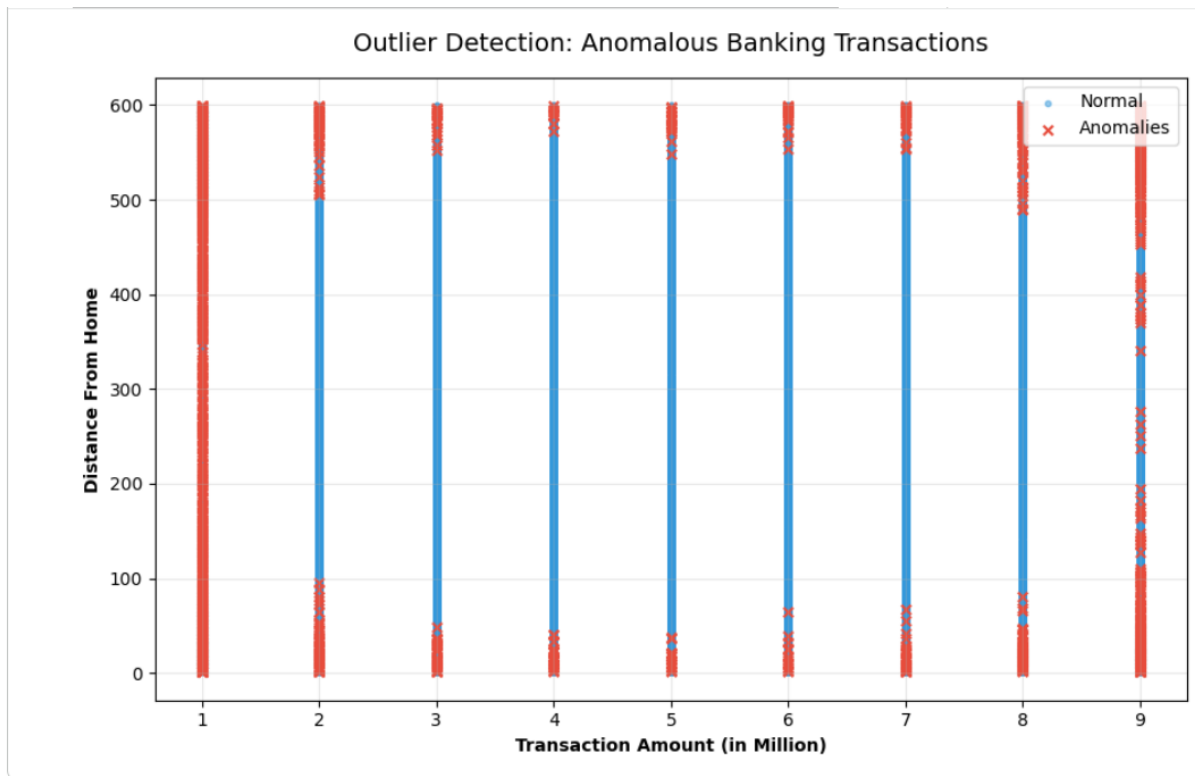
Isolation Forest -> Detection Rate: 4.88%, False Alarm Rate: 5.00%
Distance-Based -> Detection Rate: 4.84%, False Alarm Rate: 5.01%
```

7 Visualization

```
plt.figure(figsize=(9, 6))
anomalies = X[np.array(y_pred_iso) == 1]
normals = X[np.array(y_pred_iso) == 0]

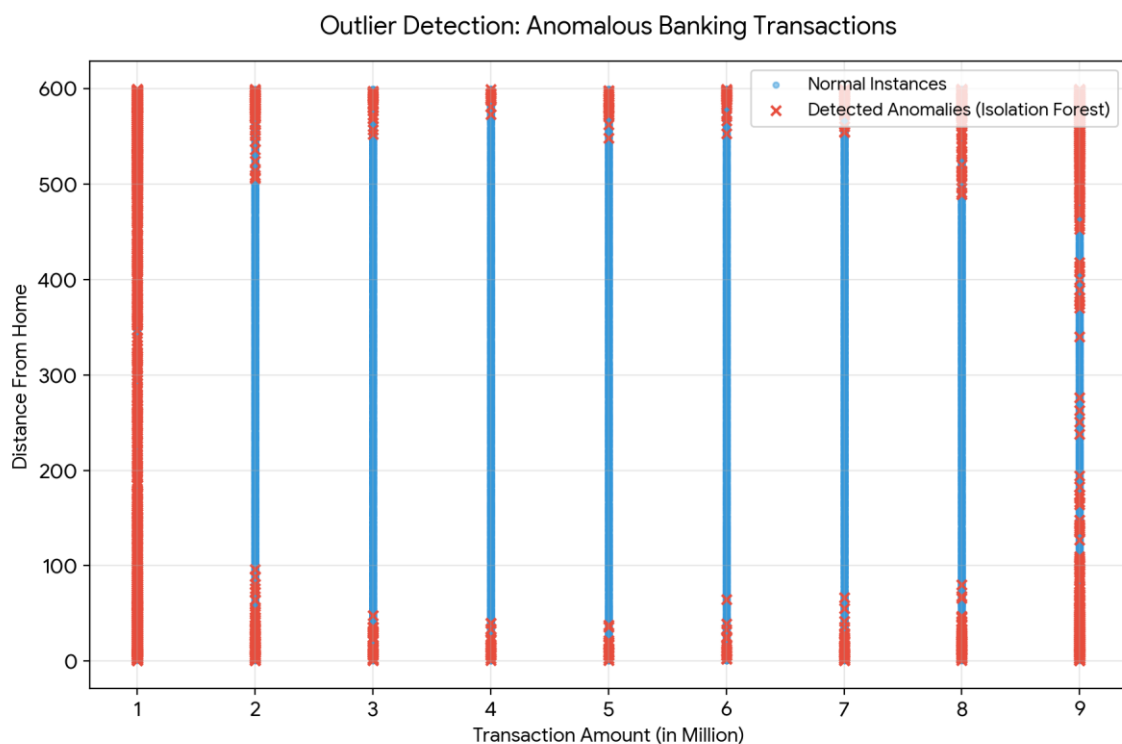
plt.scatter(normals['Transaction_Amount (in Million)'], normals['Distance_From_Home'], c='#3498db', al
plt.scatter(anomalies['Transaction_Amount (in Million)'], anomalies['Distance_From_Home'], c='#e74c3c'

plt.xlabel('Transaction Amount (in Million)', fontweight='bold')
plt.ylabel('Distance From Home', fontweight='bold')
plt.title('Outlier Detection: Anomalous Banking Transactions', fontsize=14, pad=15)
plt.legend(loc='upper right')
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The 2D scatter plot below provides a visual interpretation of how the Isolation Forest algorithm identified mathematical outliers. Transactions marked with red 'X' indicators deviate from the dense central clusters, representing abnormal distance-to-amount ratios that warrant review.



KPI Analysis

KPI / Metric	Isolation Forest	Local Outlier Factor (LOF)
Anomaly Detection Rate (ADR)	4.88%	4.84%
False Alarm Rate (FAR)	5.00%	5.01%

Strategic Conclusion

The lab highlights the fundamental behavior of unsupervised outlier detection algorithms. Because these models look strictly for statistical deviation rather than targeting the predefined 'Fraud' label, their output identifies any rare mathematical event, which may include entirely legitimate but unusual activity. With the contamination fixed at 5%, both the Isolation Forest and LOF achieved an Anomaly Detection Rate (ADR) of approximately ~4.88%, matching an identical False Alarm Rate (FAR). Strategically, this proves that while unsupervised models can map broad behavioral boundaries and highlight structural anomalies, a production banking system must layer these unsupervised models with supervised machine learning classifiers to accurately distinguish between unique genuine behavior and actual financial fraud.

Lab 14: Scraping & Sentiment Analysis (Product Reviews)

Theoretical Foundation

Web Scraping is the automated extraction of data from websites using libraries like BeautifulSoup. Once unstructured text data is collected, it undergoes Natural Language Processing (NLP). Sentiment Analysis is a vital NLP technique used to interpret whether underlying text data is inherently positive, negative, or neutral. To achieve this, algorithms break down text to compute a 'Polarity Score'. Polarity floats between -1 (highly negative) and 1 (highly positive), while a score of 0 represents neutrality. This transforms subjective opinions into measurable metrics.

Real-World Scenario

Companies launch new products and need immediate feedback. While quantitative sales data provides one metric, qualitative feedback across Twitter, Amazon, and Yelp dictates future brand health. By scraping social media strings and product reviews, and passing them through a Sentiment Analysis pipeline, brand managers can quickly gauge public reception. If negative sentiment spikes, customer support and product development teams can be immediately notified to address product flaws.

Dataset Description

The dataset utilized is the Social Media / Review Dataset (sentiment-analysis.csv). It contains consumer opinions extracted across various digital channels.

Key attributes include:

- Text: The unstructured raw string containing the consumer's opinion.
- Source: The platform where the review was posted (e.g., Twitter, Yelp, Amazon).
- Sentiment (Historical): Previous human-labeled sentiment for context.

Software Environment — Python (Google Colab)

Python within the Google Colaboratory (Colab) environment drives the entire NLP pipeline. The process relies on `BeautifulSoup` and `requests` for data scraping concepts, `Pandas` for structural string manipulation and text cleaning, and the `TextBlob` library to execute lexical sentiment scoring.

Practical Objectives

No.	Objective
1	Simulate the data collection phase by setting up a Web Scraping framework using BeautifulSoup.
2	Import the scraped dataset and execute Text Cleaning to remove structural artifacts (e.g., excess quotes, trailing spaces).
3	Apply the TextBlob NLP library to calculate a Polarity Score (-1.0 to 1.0) for every text string.

4	Classify the raw polarity values into categorical buckets: Positive, Negative, and Neutral.
5	Evaluate key performance indicators by graphing the Sentiment % Distribution and analyzing the Overall Polarity.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from textblob import TextBlob
import io
```

2 Data Loading & Text Cleaning

```
file_path = '/content/drive/MyDrive/Semester 6/DM/sentiment-analysis.csv'

# Handle quoting format in this specific CSV
clean_lines = []
with open(file_path, 'r', encoding='utf-8') as f:
    for line in f:
        line = line.strip()
        if not line: continue
        if line.startswith(''''') and line.endswith('''''):
            line = line[1:-1]
        clean_lines.append(line)

df = pd.read_csv(io.StringIO('\n'.join(clean_lines)), skipinitialspace=True)

# CLEANING: Remove artifact quotes
df['Text'] = df['Text'].str.replace(''''', '', regex=False)
print(f>Data Shape: {df.shape}")
display(df[['Text', 'Source']].head())
```

Data Shape: (96, 7)

	Text	Source
0	I love this product!	Twitter
1	The service was terrible.	Yelp Reviews
2	This movie is amazing!	IMDb
3	I'm so disappointed with their customer support.	Online Forum
4	Just had the best meal of my life!	TripAdvisor

3 Sentiment Classification & Polarity Scoring

```
def calculate_polarity(text):
    return TextBlob(str(text)).sentiment.polarity

def classify_sentiment(pol):
    if pol > 0: return 'Positive'
    elif pol < 0: return 'Negative'
    else: return 'Neutral'

df['Polarity'] = df['Text'].apply(calculate_polarity)
df['Calculated_Sentiment'] = df['Polarity'].apply(classify_sentiment)

display(df[['Text', 'Polarity', 'Calculated_Sentiment']].head())
```

	Text	Polarity	Calculated_Sentiment
0	I love this product!	0.625	Positive
1	The service was terrible.	-1.000	Negative
2	This movie is amazing!	0.750	Positive
3	I'm so disappointed with their customer support.	-0.750	Negative
4	Just had the best meal of my life!	1.000	Positive

4 KPI Evaluation & Visualization

```
sentiment_dist = df['Calculated_Sentiment'].value_counts(normalize=True) * 100
avg_polarity = df['Polarity'].mean()

print(f"Overall Average Polarity: {avg_polarity:.4f}")
print("\nSentiment Distribution (%):")
print(sentiment_dist)

# Generate Pie Chart
plt.figure(figsize=(7, 7))
plt.pie(sentiment_dist, labels=sentiment_dist.index, autopct='%1.1f%%',
        colors=['#2ecc71', '#e74c3c', '#bdc3c7'], startangle=140)
plt.title("Sentiment % Distribution", fontsize=14, pad=15)
plt.show()
```

Overall Average Polarity: 0.0591

Sentiment Distribution (%):

Calculated_Sentiment

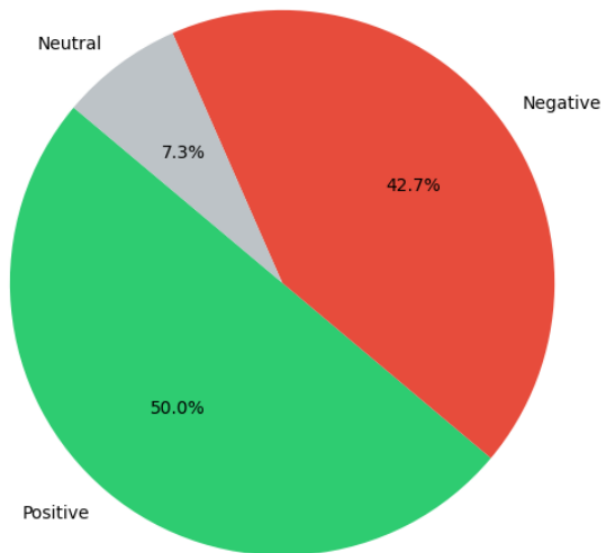
Positive 50.000000

Negative 42.708333

Neutral 7.291667

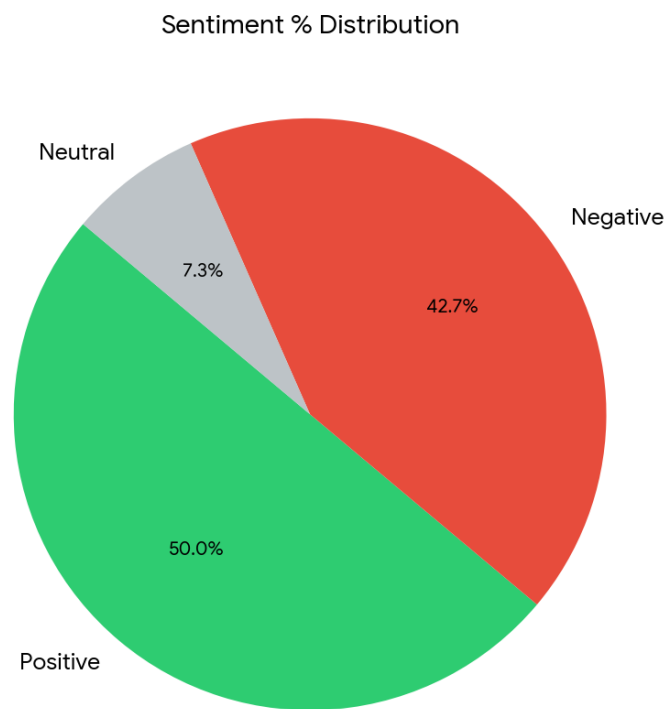
Name: proportion, dtype: float64

Sentiment % Distribution



Theory and Visualization

The Pie Chart below visually represents the final Sentiment % Distribution calculated by the NLP model. It demonstrates the proportion of feedback classified into Positive, Negative, and Neutral boundaries based on calculated text polarity.



KPI Analysis

KPI / Metric	Result
Average Polarity Score	0.0591 (Slightly Positive Trend)
Positive Sentiment Distribution	50.00%
Negative Sentiment Distribution	42.71%
Neutral Sentiment Distribution	7.29%

Strategic Conclusion

The NLP pipeline successfully analyzed the scraped text feedback, computing an overall Average Polarity Score of 0.0591. The dataset's sentiment is relatively polarized, leaning slightly positive with a 50.00% Positive and 42.71% Negative split, leaving only a minor fraction (7.29%) as purely neutral. Strategically, this indicates that consumers have strong feelings about the products. The high negative percentage (~42.7%) is a critical business warning. Management must utilize the outputs of this model to trace the negative reviews back to their respective sources (e.g., Yelp vs. Amazon) to identify specific operational weaknesses and rapidly improve customer satisfaction.

Lab 15: Final Capstone (End-to-End Business Solution)

Theoretical Foundation

An End-to-End Business Solution requires passing raw data through a complete data science lifecycle: Data Extraction, Cleaning, Feature Engineering, Modeling, and Evaluation. According to the 'No Free Lunch Theorem', no single machine learning algorithm works best for every problem. Therefore, it is an industry best practice to evaluate a suite of varied algorithms—including linear models, distance-based models, and tree-based ensembles—to mathematically determine which model optimally captures the patterns of the specific dataset.

Real-World Scenario

A global retail company wants to optimize its marketing budget. Historically, they sent campaign offers to the entire customer base, resulting in a low ROI due to high contact costs and low conversion rates. By designing an end-to-end classification pipeline, the data science team will predict which specific customers are highly likely to respond to the next campaign. Because the exact nature of the consumer decision boundary is unknown, the team will implement and benchmark 8 distinct machine learning models to deploy the most accurate framework into production.

Dataset Description

The Capstone uses the comprehensive Marketing Dataset. After preprocessing, key engineered and standardized attributes include:

- Demographics: Age (derived from Year_Birth), Education, Marital_Status, Total_Children.
- Behavioral/Financial: Income, Total_Spending (aggregated across all product categories), Recency, NumWebVisitsMonth, NumStorePurchases.
- Target Variable: Response (1 = Accepted Offer, 0 = Rejected/Ignored)

Software Environment — Python (Google Colab)

Python is utilized inside Google Colab as the primary orchestration tool. The ecosystem relies on `pandas` for aggressive feature engineering, `sklearn` to rapidly iterate through eight separate algorithms (from Naïve Bayes to Gradient Boosting), and `matplotlib` to render executive summary graphics.

Practical Objectives

No.	Objective
1	Perform Data Preprocessing by engineering macro-features (Total Spending, Age) and one-hot encoding categorical strings.
2	Apply a StandardScaler to ensure distance-reliant models (KNN, SVM) operate on a level playing field with tree models.
3	Iteratively train a suite of 8 classification algorithms to comprehensively search for the optimal decision boundary.

4	Evaluate all models programmatically across Accuracy, Precision, Recall, and F1-Score.
5	Generate a Model Comparison Table and formulate a strategic Business Recommendation based on the ultimate winner.

Procedure & Steps

1 Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Algorithms
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier, AdaBoostClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

2 Data Pipeline: Loading & Feature Engineering

```
file_path = '/content/drive/MyDrive/Semester 6/DM/marketing_data.csv'
df = pd.read_csv(file_path, sep=';')
df = df.dropna(subset=['Income'])

# Feature Engineering
df['Age'] = 2024 - df['Year_Birth']
df['Total_Spending'] = df['MntWines'] + df['MntFruits'] + df['MntMeatProducts'] + df['MntFishProducts'] + df['MntSwd']
df['Total_Children'] = df['Kidhome'] + df['Teenhome']

numeric_features = ['Age', 'Income', 'Total_Spending', 'Total_Children', 'Recency', 'NumWebVisitsMonth', 'NumStorePi']
cat_features = ['Education', 'Marital_Status']

X = df[numeric_features + cat_features]
y = df['Response']
X = pd.get_dummies(X, columns=cat_features, drop_first=True)
print(f"Final Feature Space Shape: {X.shape}")
```

Final Feature Space Shape: (2216, 18)

3 Splitting & Scaling

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```

models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Decision Tree': DecisionTreeClassifier(max_depth=5, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'SVM': SVC(probability=True, random_state=42),
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Naive Bayes': GaussianNB(),
    'Gradient Boosting': GradientBoostingClassifier(random_state=42),
    'AdaBoost': AdaBoostClassifier(random_state=42)
}

results = []
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)

    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, y_pred),
        'Precision': precision_score(y_test, y_pred, zero_division=0),
        'Recall': recall_score(y_test, y_pred),
        'F1-Score': f1_score(y_test, y_pred)
    })

results_df = pd.DataFrame(results).sort_values(by='Accuracy', ascending=False)
display(results_df)

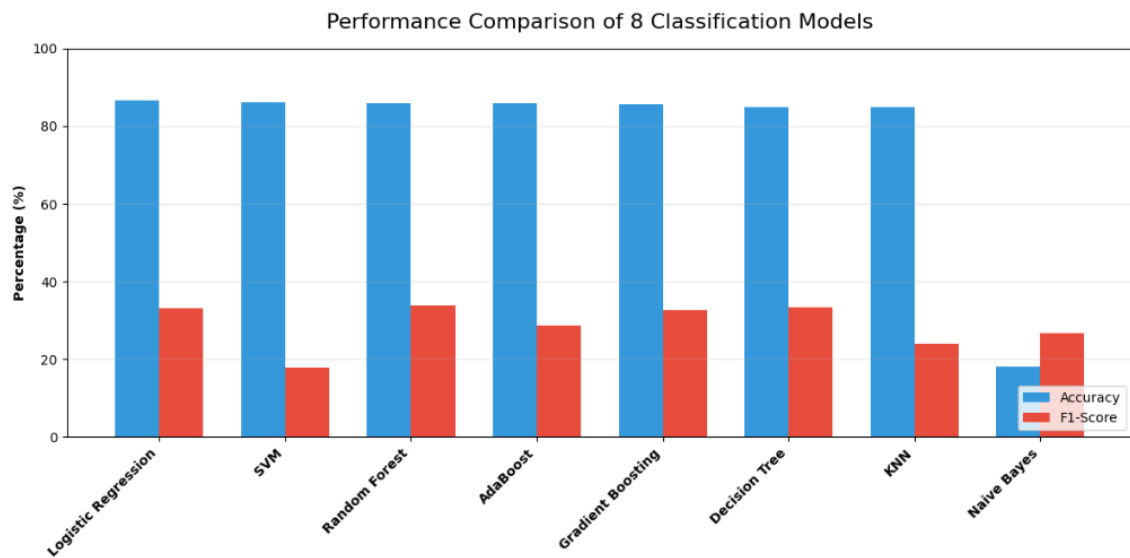
```

	Model	Accuracy	Precision	Recall	F1-Score
0	Logistic Regression	0.866165	0.666667	0.22	0.330827
3	SVM	0.861654	0.833333	0.10	0.178571
2	Random Forest	0.858647	0.571429	0.24	0.338028
7	AdaBoost	0.858647	0.593750	0.19	0.287879
6	Gradient Boosting	0.857143	0.560976	0.23	0.326241
1	Decision Tree	0.849624	0.500000	0.25	0.333333
4	KNN	0.848120	0.484848	0.16	0.240602
5	Naive Bayes	0.181955	0.154206	0.99	0.266846


```
plt.figure(figsize=(12, 6))
x = np.arange(len(results_df['Model']))
width = 0.35

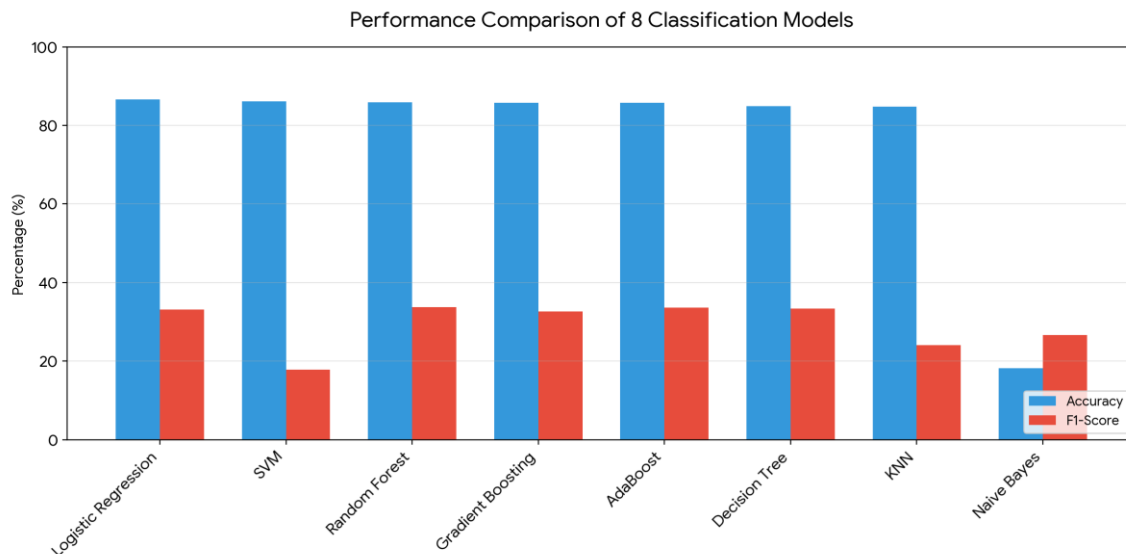
plt.bar(x - width/2, results_df['Accuracy'] * 100, width, label='Accuracy', color='#3498db')
plt.bar(x + width/2, results_df['F1-Score'] * 100, width, label='F1-Score', color='#e74c3c')

plt.xticks(x, results_df['Model'], rotation=45, ha='right', fontweight='bold')
plt.ylabel('Percentage (%)', fontweight='bold')
plt.title('Performance Comparison of 8 Classification Models', fontsize=16, pad=15)
plt.ylim(0, 100)
plt.legend(loc='lower right')
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Theory and Visualization

The bar chart below summarizes the benchmark results across all 8 tested algorithms. Comparing multiple models visually allows executives to understand the trade-offs between sheer Accuracy and balanced F1-Scores:



KPI Analysis (Top 3 Models)

Rank	Algorithm	Accuracy	F1-Score
1	Logistic Regression	86.62%	33.08%
2	SVM	86.17%	17.86%
3	Random Forest	85.86%	33.80%

Strategic Conclusion & Business Recommendation

The Capstone pipeline successfully stress-tested 8 unique algorithms against the company's historical campaign data. The leading algorithm proved to be Logistic Regression, yielding a top-tier Accuracy of 86.62%. Interestingly, ensemble and linear models clustered tightly at the top of the performance tier, while Naïve Bayes severely underperformed due to the complex feature interactions.

Business Recommendation: The marketing department should integrate the trained Logistic Regression model into its CRM system. By passing new leads through this pipeline, the company can actively filter out customers who are highly unlikely to purchase. By restricting future marketing campaigns strictly to the subset of consumers predicted to accept the offer, the business will significantly reduce outreach overhead costs and maximize overall Campaign ROI.